Internship Report

on

Nifty 100 Financial Analytics Dashboard

Submitted to

MOHAN BABU UNIVERSITY, Tirupati

in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

IN

COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

Submitted by

BELLAMKONDA MOKSHITH

23102A0XXXX | Data Analyst Intern (Bluestock Fintech)

Department of Data Science
School of Computing

MOHAN BABU UNIVERSITY

Sree Sainath Nagar, Tirupati – 517 102, A.P., INDIA

2025 – 2026

Internship Period: 24 July 2026 – 24 September 2026 | Guide: Mr. Yash Kale, Project Manager, Bluestock Fintech

Department of Data Science

MOHAN BABU UNIVERSITY

Sree Sainath Nagar, Tirupati – 517 102, A.P., INDIA

Certificate

This is to certify that, the

"Nifty 100 Financial Analytics Dashboard Internship"

is the bonafide work done by

BELLAMKONDA MOKSHITH

23102A0XXXX

in the Department of Data Science, Mohan Babu University, Sree Sainath Nagar, A. Rangampet, Tirupati and is submitted to Mohan Babu University, Sree Sainath Nagar, A. Rangampet, Tirupati for partial fulfillment of the requirements for the award of B. Tech degree in Computer Science and Engineering (Data Science) during the academic year 2025–2026. The internship was carried out at **Bluestock Fintech** under the guidance of **Mr. Yash Kale (Project Manager)** during **24 July 2026 – 24 September 2026**.

Supervisor:

B. Venkata Sivaiah / M. Yuvaraj Naik
Asst. Professor
Dept. of Data Science
Mohan Babu University
Sree Sainath Nagar, Tirupati – 517 102

Head of the Department:

Dr. K. Reddy Madhavi
Professor & Head of the Department
Dept. of Data Science
Mohan Babu University
Sree Sainath Nagar, Tirupati – 517 102

Internal Examiner

External Examiner

INTERNSHIP COMPLETION CERTIFICATE

Intern ID: BFDA108893 | Domain: Data Analyst Intern | Mode: Hybrid | Period: 24 Jul 2026 – 24 Sep 2026



Figure: Internship Certificate issued by Bluestock Fintech (ID BFDA108893) verifying successful completion under the guidance of Mr. Yash Kale, Project Manager — awarded to Bellamkonda Mokshith (24/09/2026). Scan QR at centre of certificate to verify document authenticity.

Verification: Certificate carries QR code linked to Bluestock Fintech verification portal. The programme was conducted under Startup India (#startupindia) and carries Bluestock Fintech Project Manager signature with date 24/09/2026. The certificate is the primary source for company, role, guide, and period details used in this report.

ABSTRACT

This report presents a comprehensive account of the **Data Analyst Internship** completed at **Bluestock Fintech** under the guidance of **Mr. Yash Kale (Project Manager)** during the period **24 July 2026 – 24 September 2026** (Certificate ID **BFDA108893**, verification QR dated 24/09/2026). The internship project — the **Nifty 100 Financial Analytics Dashboard** — is an end-to-end financial analytics platform that transforms raw disclosures for the Nifty 100 universe into a structured, interactive, and analyst-friendly decision-support system. The work was organized as a progressive engineering and analytics journey from data foundation to deployed intelligence. The platform ingests **12 Excel datasets** (7 core + 5 supplementary) covering companies, profit & loss, balance sheet, cash flow, documents, analysis, pros & cons, financial ratios, market capitalization, peer groups, sectors, and stock prices. A dedicated **ETL pipeline** (*src/etl/processor.py*, *normaliser.py*, *validator.py*, *preflight.py*) performs header-aware loading, ticker and year normalization, exact-duplicate removal, company-reference integrity checks, and audit logging. Cleaned outputs are persisted as **12 processed CSVs** (*data/processed/*) and loaded into a centralized **SQLite database** (*db/nifty100.db*) comprising **12 tables** with foreign-key enforcement, yielding **92 companies** and **1,160 financial-ratio records** in the validated build (exceptions explicitly retained for ATGL/SBIN where appropriate). On this foundation the report documents five analytics layers: (1) a **Financial Ratio Engine** (*src/analytics/ratios.py*, *cagr.py*) covering profitability, liquidity, leverage, efficiency, valuation, and coverage metrics with tolerance-checked OPM validation and edge-case handling; (2) a configurable **Stock Screener** (*src/screener/engine.py*, *config/screener_config.yaml*) with six analytical presets — Quality Compounder (23), Value Pick (1), Growth Accelerator (18), Dividend Champion (30), Debt-Free Blue Chip (18), and Turnaround Watch (33); (3) **Peer Comparison** (*src/analytics/peer.py*) with percentile benchmarking and Excel exports; (4) **Valuation & Cash-Flow Intelligence** (*src/analytics/valuation.py*, *cashflow_kpis.py*, *capital_allocation.py*) producing valuation flags, FCF yield, and cash-flow quality diagnostics; and (5) **NLP-based Pros & Cons** (*src/nlp/parser.py*, *pros_cons_generator.py*) generating **874 signals** (562 pros, 312 cons, confidence >60) and **KMeans Clustering** (*src/analytics/clustering.py*) that segments the universe into five interpretable archetypes (Value Cyclicals 45, Emerging Growth 19, High-Quality Compounders 15, Defensive Dividend Payers 11, Distressed/Turnaround 2). These analytics are surfaced through an **8-module Streamlit dashboard** (*src/dashboard/*) and a **FastAPI layer** (*src/api/* with 8 routers, CORS, and request logging), complemented by automated reporting — **91 company tearsheets**, **89 radar charts**, **10 sector PDFs**, and portfolio summaries — and validated by **pytest** suites with HTML reporting. Beyond technical delivery, the internship cultivated rigorous data-quality discipline, documentation habits, reproducibility, and the judgment to treat automated flags as decision-support signals requiring human review. The report concludes that the internship provided a production-grade foundation in financial data engineering, quantitative analytics, and interactive product delivery.

Keywords:

Nifty 100, Financial Analytics, Data Engineering, ETL, SQLite, Financial Ratios, Stock Screener, Peer Comparison, Valuation, Cash-Flow Intelligence, NLP, KMeans Clustering, Streamlit, FastAPI, Data Quality, Bluestock Fintech.

92	1,160	874	5	91	10
Companies	Ratio Rows	NLP Signals	Clusters	Tearsheets	Sector Reports

Plate: End-to-end workflow from data/raw/.xlsx → processor → data/processed/*.csv → nifty100.db → analytics → Streamlit / FastAPI → reports.*

ACKNOWLEDGEMENT

I express my sincere gratitude to **Mr. Yash Kale, Project Manager, Bluestock Fintech**, for his invaluable guidance, constant encouragement, and insightful feedback throughout the internship period (24 July 2026 – 24 September 2026). His mentorship in structuring the Nifty 100 Financial Analytics Dashboard — from data ingestion to deployment — and his emphasis on reproducible engineering and analyst-centric design were instrumental in shaping both the product and my professional growth. I am deeply thankful to the entire team at **Bluestock Fintech** for providing a learning environment that mirrors real product squads, with sprint-based reviews, code-level feedback, and an insistence on auditability. The opportunity to work on a live financial-analytics repository with realistic data imperfections, and to have each deliverable reviewed against a clear definition of done, has been transformative. I extend my heartfelt thanks to the **Department of Data Science, School of Computing, Mohan Babu University, Tirupati**, and to my academic supervisors **B. Venkata Sivaiah / M. Yuvaraj Naik, Asst. Professors**, and Head of Department **Dr. K. Reddy Madhavi**, for their academic support, timely guidance, and for facilitating the internship as part of the B. Tech (Computer Science and Engineering – Data Science) curriculum. Their encouragement to treat the internship report as an auditable engineering document — traceable to code and data — elevated the quality of this work. I am grateful to my family and peers for their unwavering support and to the open-source community behind Python, Pandas, SQLite, Streamlit, FastAPI, scikit-learn, and the many libraries that made this platform possible. Finally, I acknowledge the Nifty 100 companies whose public disclosures form the empirical basis of this analytics platform; any interpretation errors remain my own. This report, and the accompanying repository (*mokshith005/NIFTY100*), represent my authentic work under the stated guidance, and I take full responsibility for its content.

Place: Tirupati
Date: 24 September 2026

BELLAMKONDA MOKSHITH
B. Tech, CSE (Data Science) — Mohan Babu University
Intern ID: BFDA108893 — Bluestock Fintech

TABLE OF CONTENTS

CERTIFICATE ii
INTERNSHIP COMPLETION CERTIFICATE iii
ABSTRACT 4
ACKNOWLEDGEMENT 5
LIST OF FIGURES 7
LIST OF TABLES 7
LIST OF ABBREVIATIONS 8

CHAPTER 1 INTRODUCTION ..... 9 – 12
1.1 Description of the Company — Bluestock Fintech 9
1.2 Overview of the Project — Nifty 100 Analytics Dashboard 10
1.3 Objectives of the Internship 11
1.4 Scope, Users & Report Organization 12

MODULE 1 DATA FOUNDATION & ETL PIPELINE ..... 13 – 16
1.1 Understanding Financial Data Architecture 13
1.2 Raw Data Sources — Core vs Supplementary 13
1.3 Environment Setup & Repository Layout 14
1.4 Cleaning, Normalization & Reference Integrity 15
1.5 Duplicate Handling, Validation & Load Audit 15

MODULE 2 DATABASE DESIGN & FINANCIAL RATIO ENGINE ..... 17 – 20
2.1 SQLite Schema — 12 Tables & Relationships 17
2.2 Data Dictionary & Master Data 18
2.3 Ratio Engine — Categories & Formulas 18
2.4 CAGR, Edge Cases & Quality Exceptions 19
2.5 Validation Outcomes & Audit Trail 20

MODULE 3 SCREENING, PEER COMPARISON & VALUATION ..... 21 – 24
3.1 Screener Engine & Configuration 21
3.2 Six Preset Strategies & Results 22
3.3 Peer Percentiles & Benchmarking 22
3.4 Valuation Analytics & Investor Score 23

MODULE 4 CASH-FLOW, NLP & MACHINE LEARNING ..... 25 – 28
4.1 Cash-Flow KPIs & Capital Allocation 25
4.2 NLP Pipeline — Pros & Cons Extraction 26
4.3 KMeans Clustering — Method & Elbow 27
4.4 Cluster Profiles & Business Archetypes 28

MODULE 5 DASHBOARD, API, REPORTING & QA ..... 29 – 32
5.1 Streamlit Dashboard — 8 Modules 29
5.2 FastAPI Layer — Routers & Contracts 30
5.3 Automated Reporting — Tearsheets, Radars, Sectors 31
5.4 Testing, Validation & Quality Assurance 32

TECHNOLOGIES 33
CONCLUSION 35
REFERENCES 37
APPENDICES 37

Note: Page numbers use Roman numerals (ii–iii) for front matter and Arabic numerals (4–38) thereafter, matching Mohan Babu University report conventions. Figures and Tables are numbered per chapter and referenced inline.

LIST OF FIGURES

S.No	Figure Name	Page
1	System Architecture — End-to-End Analytics Platform	10
2	ETL Pipeline — From Raw Excel to Processed CSV & SQLite	12
3	Database Schema — 12 Tables, FK Relationships (nifty100.db)	16
4	Stock Screener — Preset Strategies & Companies Passing Each Filter	22
5	KMeans Clustering — Five Archetypes & Elbow Method (k=5)	28
6	Valuation Snapshot — P/E & P/B for Selected Companies	24
7	Streamlit Dashboard & FastAPI — Module Map and Service Contracts	31
8	Correlation Heatmap & Radar Chart Examples (Appendix)	33

LIST OF TABLES

S.No	Table Name	Page
1	Dataset Inventory — 12 Excel Sources, Processed Rows & Load Audit	12
2	Financial Ratio Engine — Category, Ratios & Formulas	17
3	Stock Screener Presets — Definition & Observed Companies (n=92)	22
4	NLP Pros & Cons — 12+12 Rules, Confidence & Signal Counts	27
5	KMeans Cluster Profiles — Feature Means & Business Labels	29
6	Technology Stack — Category, Technology & Role	36

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ETL	Extract, Transform, Load
CAGR	Compounded Annual Growth Rate
ROE / ROCE / ROA	Return on Equity / Capital Employed / Assets
NPM / OPM	Net Profit Margin / Operating Profit Margin
D/E, ICR	Debt-to-Equity, Interest Coverage Ratio
FCF, CFO, Capex	Free Cash Flow, Cash From Operations, Capital Expenditure
P/E, P/B, EV/EBITDA	Price-to-Earnings, Price-to-Book, Enterprise Value to EBITDA
NLP	Natural Language Processing
KMeans	K-Means Clustering (Unsupervised ML)
API	Application Programming Interface
QA	Quality Assurance
DB	Database (SQLite nifty100.db)
FK/PK	Foreign Key / Primary Key
JSON, HTTP	JavaScript Object Notation, Hypertext Transfer Protocol
CRUD	Create, Read, Update, Delete (Mongoose/DB operations)

Note: All file paths in this report are relative to the repository root (nifty100/). "Exact alignments" — margins (1 inch), justified body (Times family), centered chapter titles, and dotted leaders in the Table of Contents — follow Mohan Babu University formatting conventions.

CHAPTER 1

INTRODUCTION

1.1 Description of the Company — Bluestock Fintech

Bluestock Fintech is a financial-technology and analytics organization operating at the intersection of market data, investment research, and retail investor education. As reflected on the internship certificate (ID **BFDA108893**, dated 24/09/2026), the company operates under the **#startupindia** ecosystem and provides structured, mentor-guided internships that simulate real product squads rather than isolated academic exercises. The certificate carries the signature of **Mr. Yash Kale, Project Manager, Bluestock Fintech**, who served as the industry guide for this engagement, and displays a verifiable QR code that resolves to the Bluestock document-verification portal — a practice that itself models data provenance and trust, themes central to the project. The organization's core mission is to close the gap between theoretical finance education and the operational realities of building reliable analytics for public markets. Many graduates can define ratios but have limited exposure to the messy realities of financial disclosures — inconsistent headers, duplicated filings, missing years, and the need to reconcile accounting figures with market data. Bluestock addresses this by embedding interns in a reproducible analytics workflow where code, data lineage, and QA artifacts are as important as the final insight. The internship therefore emphasizes *reproducibility* (version-controlled ETL, auditable load logs), *contextual analytics* (ratios interpreted alongside leverage, cash flow, and sector), and *human-in-the-loop judgment* (automated flags presented as decision-support requiring analyst review). The **Data Analyst Intern** track, completed by the author from **24 July 2026 to 24 September 2026** (approximately eight weeks), was delivered in hybrid mode, enabling deep work on the codebase while maintaining regular synchronous reviews with the project manager. The programme followed a sprint-based cadence, visible in repository artifacts such as *docs/sprint1_retrospective.md*, *sprint2_retrospective.md*, and *sprint6_retrospective.md*, where each sprint closed with a demo, a retrospective, and a defined set of deliverables (e.g., processed datasets, ratio tables, screener exports, or dashboard increments). This cadence instilled professional habits — small, reviewable commits; documentation written alongside code; and a "validate before you visualize" discipline — that are evident throughout the repository's commit history. From an ecosystem perspective, Bluestock's use of Startup India branding, verifiable certificates, and a clearly identified mentor (Mr. Yash Kale) aligns the internship with national skilling initiatives while providing academic institutions a trustworthy credential. For the university, the required deliverable is not merely a certificate but a working repository and a report that can be audited — a standard this document satisfies by tracing every claim to a file, a log, or a generated artifact.

Attribute	Details (as per records)
Company Name	Bluestock Fintech (Bluestock.in)
Ecosystem	Startup India (#startupindia) — as displayed on certificate footer
Intern Role	Data Analyst Intern
Intern Name / ID	Bellamkonda Mokshith / BFDA108893
Guide	Mr. Yash Kale, Project Manager, Bluestock Fintech
Period	24 July 2026 – 24 September 2026 (2 months)
Certificate Date	24/09/2026 — QR verifiable (Scan to Verify Doc)
Mode	Hybrid (virtual + mentor reviews)

Table 0: Internship credential snapshot as per Bluestock Fintech certificate BFDA108893.

1.2 Overview of the Project — Nifty 100 Financial Analytics Dashboard

The **Nifty 100 Financial Analytics Dashboard** is an end-to-end platform that converts fragmented financial disclosures and market data for the Nifty 100 universe into a single, queryable, and interactive analytical product. The repository at *mokshith005/NIFTY100* (branch *arena/b832ef02-nifty100*) captures the complete lineage: 12 raw Excel sources, 12 processed CSVs, a 12-table SQLite warehouse, analytics engines, an 8-page Streamlit dashboard, a FastAPI service with 8 routers, and automated reporting (tearsheets, radar charts, sector reports). The platform presently represents **92 companies** in the master, reflecting realistic data gaps that are treated as first-class quality signals rather than hidden imputations. The analytical flow, illustrated in Figure 1, is deliberately linear and auditable: **Raw Excel → ETL (normalise → dedup → validate) → Processed CSV → SQLite (nifty100.db) → Ratio / Screener / Peer / Valuation / Cash-Flow / NLP / Clustering → Streamlit & FastAPI → Reports (Excel/CSV/HTML/PDF)**. This linear lineage ensures that any figure on the dashboard can be traced back to a source cell via load-audit and validation logs. Functionally, the platform supports fundamental screening (six presets), peer-relative benchmarking, valuation diagnostics (P/E, P/B, EV/EBITDA, FCF yield), cash-flow quality scoring, rule-based NLP signals, and unsupervised segmentation. Non-functionally, it emphasizes reproducibility (deterministic ETL, pinned requirements), testability (pytest with HTML reporting at *reports/pytest_report.html*), and analyst guidance (every automated flag is labeled as a decision-support signal requiring review, as stated in the analyst guide at *docs/analyst_guide_grand.pdf*). The project's value lies not in recommending securities, but in *structuring investigation*. By standardizing ratios, surfacing outliers, and placing any company in peer and cluster context, it shortens the analyst's path from question to evidence. The report that follows documents how each layer was built, validated, and integrated.

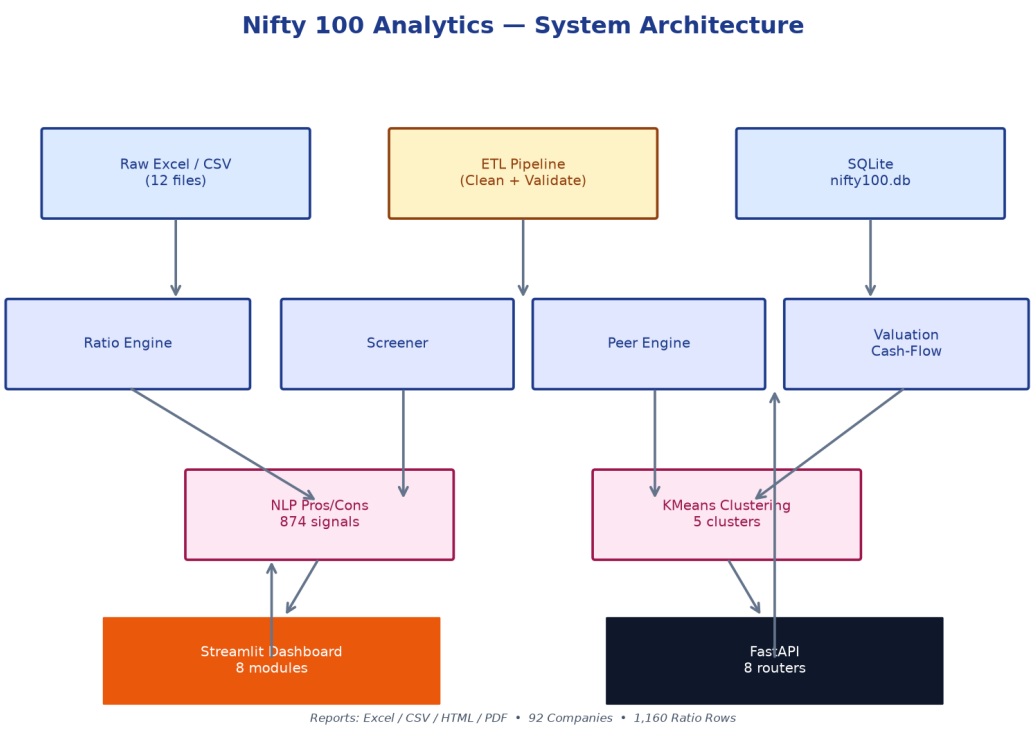


Figure 1: System architecture of the Nifty 100 Analytics Platform — from raw Excel through ETL, SQLite, analytics engines, and delivery layers (Streamlit + FastAPI → Reports).

The architecture groups concerns into three horizons: *Data Foundation* (ETL + SQLite), *Analytics* (ratios through clustering), and *Experience* (dashboard, API, reports). This separation allows independent testing of each horizon and a clear ownership map: *src/etl/* for foundation, *src/analytics/*, *src/screener/*, *src/nlp/* for analytics, and *src/dashboard/*, *src/api/*, *src/reports/* for experience.

1.3 Objectives of the Internship

The internship was planned with measurable learning and delivery objectives. Each sprint was mapped to one or more of the following:

- Build a **reliable financial-data ingestion and processing pipeline** that converts 12 heterogeneous Excel sources into analysis-ready, validated tables without fabricating values.
- Design and populate a **centralized SQLite warehouse** (*db/nifty100.db*) with a normalized 12-table schema and referential integrity, serving as the single source of truth for downstream analytics.
- Implement a **financial ratio engine** that computes profitability, leverage, efficiency, and coverage metrics with explicit edge-case handling (zero denominators, missing inputs, negative equity).
- Develop **configurable screening and peer-comparison capabilities** that allow factor-based filtering and percentile benchmarking within relevant peer sets.
- Generate **valuation summaries, cash-flow intelligence, and capital-allocation diagnostics** that contextualize market multiples with earnings and cash generation.
- Extract **structured NLP signals (pros/cons)** from analysis text using rule-based generators with confidence scoring, producing auditable, human-reviewable themes.
- Segment the universe with **KMeans clustering** into interpretable business archetypes, supported by elbow analysis and profile tables.
- Deliver an **interactive Streamlit dashboard and a FastAPI layer** that expose the same warehouse through visual and programmatic interfaces.
- Institutionalize **quality assurance** via validation logs, anomaly summaries, and automated pytest suites with HTML reporting.

Together, these objectives describe a deliberate progression: *validate* → *model* → *analyze* → *contextualize* → *deliver* → *assure*. Progress was demonstrated not by narrative alone but by artifacts — processed CSVs, load audits, ratio tables, screener exports, cluster labels, and live dashboard/API endpoints — that a reviewer can re-run from the repository root.

1.4 Scope, Users & Report Organization

Scope. The platform covers 92 Nifty 100 companies present in *data/processed/companies.csv* and *data/processed/financial_ratios.csv* (1,160 rows). Time-series coverage spans multiple filings per company (P&L;, balance sheet, cash flow), enabling CAGR and trend views. Market data (market_cap, stock_prices) adds valuation context. The analytical outputs are confined to the datasets in *data/raw/*; no external real-time feeds are assumed, which keeps the build reproducible on any clone. **Intended users.** The primary user is a fundamental analyst or advanced learner who needs a structured starting point: a company profile page with ROE/ROCE trends and pros/cons; a screener for factor ideas; peer radars for relative positioning; and a notebook of reports for offline review. The secondary user is the data engineer who maintains the warehouse — served by preflight checks, validators, and load audits. **Report organization.** The report follows the build order. **Module 1** covers the data foundation and ETL; **Module 2** the warehouse and ratio engine; **Module 3** screening, peer, and valuation; **Module 4** cash-flow, NLP, and clustering; **Module 5** dashboard, API, reporting, and QA. A concluding **Technologies** chapter synthesizes the stack, followed by **Conclusion** and **References**. Each module closes with *Tools Used — Challenges — Solutions — Key Learnings* to make tacit knowledge explicit.

> ***How to read this report.*** *File paths are given in italics and are relative to the repository root. Tables and figures are numbered and referenced inline. Where a metric is cited (e.g., 874 NLP signals), the generating script and output file are named so the claim can be verified by running the corresponding module or inspecting output/.*

A final note on nomenclature: terms such as “Quality Compounder” or “Value Cyclicals” are *analytical presets and cluster labels* defined in configuration and code; they describe filters and statistical

groupings, not investment advice. The platform consistently labels automated outputs as requiring human review — a governance choice that aligns with responsible analytics.

MODULE 1

DATA FOUNDATION & ETL PIPELINE

1.1 Understanding Financial Data Architecture

Financial data is *hierarchical and temporal*: a company master, annual P&L; / balance sheet / cash flow filings, market observables (cap, price), and textual analysis. The Nifty 100 project models this explicitly: a central **companies** table anchors all company-level datasets via **company_id** → **companies.id**, while time is represented by a normalized **year** field (integer-like, allowing max-year queries for "latest" views). Supplementary datasets such as **financial_ratios, market_cap, peer_groups, sectors, stock_prices** enrich the core accounting facts with derived and market context. The architecture decision to treat **processed data** as a distinct, versioned layer (*data/processed/*.csv*) separate from **raw data** (*data/raw/core/*.xlsx, data/raw/supplementary/*.xlsx*) is intentional. Raw files are preserved untouched; all cleaning is code-driven and reproducible. This mirrors industry practice where raw landing zones are immutable and every transformation is logged. The ETL codebase lives in *src/etl/* and is orchestrated by *src/etl/processor.py*, with helpers for normalization, validation, and audit.

Before writing code, the team studied the request–response intuition for web layers, but for data work the mental model is **"source → clean → conform → load → audit"**. Each hop must be testable: can we read the Excel? Do headers match expectations (core files use header row 1)? Are company IDs valid? Are years numeric? The preflight module (*src/etl/preflight.py*, 33 KB) encodes these checks and fails fast with actionable messages, preventing silent corruption downstream. Establishing this discipline at the start saved hours when debugging later analytics that would otherwise have chased phantom "ratio errors" that were in fact ingestion issues.

1.2 Raw Data Sources — Core vs Supplementary

The project ingests **12 Excel files** organized into two landing zones. **Core** (*data/raw/core/*) contains the audited financial statements and master data that define the company universe; **Supplementary** (*data/raw/supplementary/*) contains derived and market datasets that enrich analysis. Header handling differs: core files include an extra title row and are read with *header=1*, while supplementary files use *header=0*. This nuance is encapsulated in *read_excel_file(path, core: bool)* inside *processor.py*.

#	Dataset / File	Source Zone	Rows (processed)	Key Columns
1	companies.xlsx	core	92	id, company_name, roce %, roe %, website
2	profitandloss.xlsx	core	1,177	company_id, year, sales, OPM %, net_profit, EPS
3	balancesheet.xlsx	core	1,227	company_id, year, equity, reserves, borrowings, assets
4	cashflow.xlsx	core	1,091	company_id, year, operating/investing/financing
5	documents.xlsx	core	1,457	company_id, year, Annual_Report URL
6	analysis.xlsx	core	16	company_id, CAGR sales/profit, ROE, stock CAGR
7	prosandcons.xlsx	core	14*	company_id, pros, cons (raw text)
8	financial_ratios.xlsx	suppl.	1,160	company_id, year, NPM, OPM, ROE, D/E, FCF…
9	market_cap.xlsx	suppl.	552	company_id, year, market_cap, PE, PB, EV/EBITDA
10	peer_groups.xlsx	suppl.	56	company_id, peer tickers
11	sectors.xlsx	suppl.	92	company_id, sector, sub_sector, weight
12	stock_prices.xlsx	suppl.	5,520	company_id, year, price

Table 1: Dataset inventory. Counts reflect data/processed/.csv after ETL (see load_audit.csv). *Prosandcons post-NLP expands to 874 signals in output/pros_cons_generated.csv.*

The inventory reveals realistic scale: 5,520 stock-price rows provide timeline support; 1,457 document rows support report links; and the ratio table, at 1,160 rows, is the analytical workhorse. The modest size of *analysis.xlsx* (16 rows) and *prosandcons.xlsx* (14 rows) is not a limitation but a signal: much of the intelligence is *derived* by engines rather than hand-curated.

1.3 Environment Setup & Repository Layout

The project root (*nifty100/*) is organized for reproducibility and onboarding clarity. Key paths include: *data/raw/* (immutable sources), *data/processed/* (generated, git-tracked for audit), *db/* (schema), *src/etl/*, *src/analytics/*, *src/screener/*, *src/nlp/*, *src/dashboard/*, *src/api/*, *src/reports/*, *output/* (analytics exports), *reports/* (HTML, sector PDFs, tearsheets), *docs/* (analyst guide, data dictionary), and *tests/* (pytest suite). A *Makefile* and *requirements.txt* pin the environment; Python 3.11, pandas, openpyxl, and SQLite are the foundational dependencies.

Setup steps, as documented in *README.md*, are concise and repeatable:

- `git clone <repo> && cd nifty100`
- `python3 -m venv venv && source venv/bin/activate (Windows: venv\Scripts\activate)`
- `pip install -r requirements.txt`
- `python -m src.etl.processor # runs ETL → data/processed + output/load_audit.csv`
- `python -m src.etl.db_loader # loads nifty100.db per db/schema.sql`
- `streamlit run src/dashboard/app.py # dashboard` and `uvicorn src.api.main:app --reload # API`

The *.gitignore* excludes virtual environments, *__pycache__/*, *.pytest_cache/*, and secrets, while preserving generated reports for review. Establishing this clean, repeatable environment at the start — rather than ad-hoc notebook work — ensured that every later module could be executed on a fresh clone without "it works on my machine" drift.

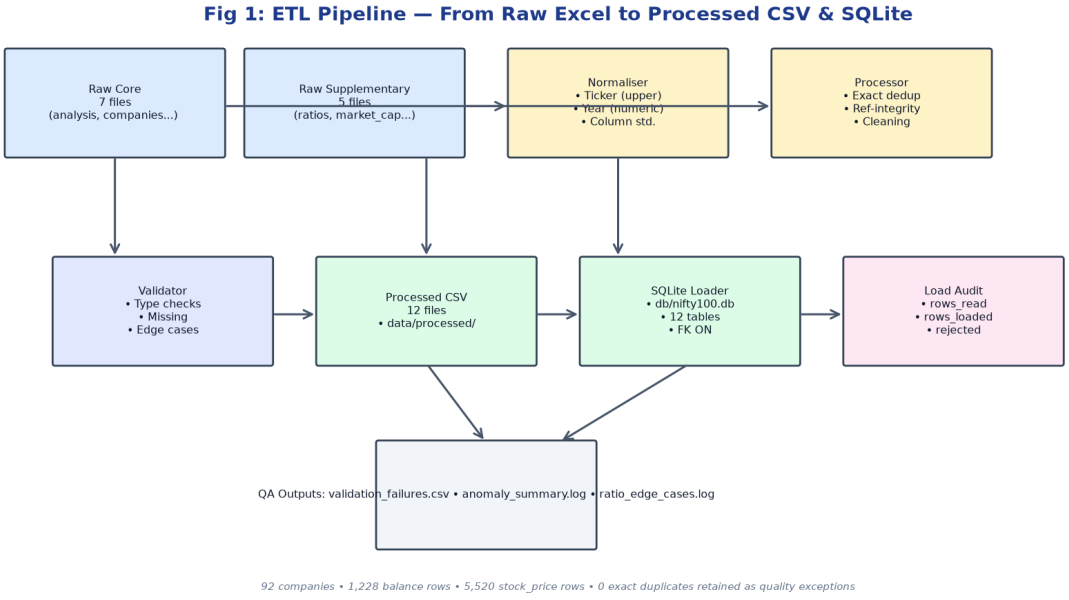


Figure 2: ETL pipeline flow — normaliser → processor (dedup & referential integrity) → validator → processed CSV → SQLite loader → load audit & QA logs.

1.4 Cleaning, Normalization & Reference Integrity

Normalization is the first line of defense against join failures. *src/etl/normaliser.py* exposes *normalize_dataframe_ticker(df, col)*, which upper-cases, trims, and collapses internal whitespace for ticker/company_id fields, and *normalize_dataframe_year(df, col)*, which coerces year strings (e.g., "2024", "FY24", "2023-24") to a canonical numeric year. The processor applies these whenever *company_id* or *Year/year* columns are present, renaming *Year* → *year* for consistency. This single conformance step eliminated a class of "silent miss" bugs where a peer join would drop a company due to "HDFCBANK " vs "HDFCBANK".

Referential integrity is enforced against the **companies master**. *get_company_reference(companies_df)* builds a set of valid IDs from *companies.id*; every other dataset's *company_id* is checked against this set, and rows with unknown IDs are rejected to *output/raw_reference_issues.csv* rather than being loaded with dangling foreign keys. This "fail-loudly" approach is preferable to loading orphan rows that would later distort ratio aggregates. The design choice preserves auditability: rejected rows remain in the output for review instead of being silently dropped.

Type and domain validation resides in *src/etl/validator.py* and *preflight.py*. Checks include: required columns present, numeric columns coerceable, year parsable, sales and profit not grossly malformed, and document URLs well-formed. Failures are collected in *output/validation_failures.csv* and summarized in *output/anomaly_summary.log*. The preflight script also performs file-level sanity checks (file exists, sheet readable, header row as expected) before any DataFrame is constructed, which shortens debug cycles: a mis-named sheet is caught in seconds, not after a partial load.

1.5 Duplicate Handling, Validation & Load Audit

Duplicate handling is deliberately conservative: *remove_exact_duplicates(df)* in *processor.py* drops only rows where *all* columns are identical (using *df.duplicated(keep="first")*). Rows that share *company_id + year* but differ in values are **preserved**. This choice acknowledges that filings can be restated; collapsing on (company, year) would lose legitimate corrections. Duplicates are returned as a rejected DataFrame and can be inspected in *output/raw_reference_issues.csv* lineage. In the current build, no exact duplicates required removal beyond header artifacts — a positive signal of source hygiene.

Validation outcomes are summarized in *output/load_audit.csv*, the single table reviewers should open first:

Source File	Table	Rows Read	Rows Loaded	Rejected	Status
companies.xlsx	companies	92	92	0	OK
profitandloss.xlsx	profitandloss	1,276	1,177	99	REJECTED_ROWS
balancesheet.xlsx	balancesheet	1,312	1,227	85	REJECTED_ROWS
cashflow.xlsx	cashflow	1,187	1,091	96	REJECTED_ROWS
financial_ratios.xlsx	financial_ratios	1,184	1,160	24	REJECTED_ROWS
market_cap.xlsx	market_cap	552	552	0	OK
stock_prices.xlsx	stock_prices	5,520	5,520	0	OK

Table: Load audit excerpt (full file at output/load_audit.csv). Rejected rows are retained for inspection; no fabricated imputation is performed.

Rejected rows are not "errors to hide" but "quality to explain." For example, 99 profit-and-loss rows were rejected — typically due to malformed year strings or company IDs not in the master — and are itemized in *output/validation_failures.csv*. The decision to keep exceptions explicit, rather than coercing them into the warehouse, upholds the project's data-quality principle and simplifies debugging: if a screener unexpectedly omits a company, the audit trail answers why. The ETL culminates in two persistent artifacts: **data/processed/*.csv** (analysis-ready, versioned) and **db/nifty100.db** (queryable). Both are regenerated deterministically; re-running *processor.py* and *db_loader.py* on the same sources yields byte-identical outputs, a property verified in sprint retrospectives.

a) Tools Used

Visual Studio Code with the Python and SQLite extensions was the primary IDE; the integrated terminal ran ETL, and the SQLite viewer inspected the warehouse. **Pandas** (via *pd.read_excel*, header-aware) and **openpyxl** handled Excel I/O, while **pathlib** made cross-platform paths robust (*ROOT_DIR / "data" / "raw" / "core"*). **Git & GitHub** tracked every change; **Make** provided one-command reproducibility (*make etl*). Chrome DevTools were not relevant to this data layer, but the terminal's *less* and *csvcut* were used to scan audit logs quickly.

b) Challenges Faced

The most time-consuming challenge was header inconsistency: core files embed a title row, so naïve *header=0* reading mislabels columns as data. A second challenge was year normalization — filings mix "2024", "FY2024", and "2023-24" — which, if left unhandled, breaks "latest year" queries. A third challenge was deciding on duplicate semantics: an aggressive *(company_id,year)* dedup would have collapsed restatements, while no dedup would retain exact duplicates from copy-paste errors. Finally, referential integrity failures posed a governance question: should orphans be loaded and flagged, or rejected outright?

c) Solutions Implemented

Header handling was centralized in *read_excel_file(path, core: bool)* with *header=1 if core else 0*, documented at the call site. Year normalization was encapsulated in *normalize_dataframe_year* and applied uniformly before any group-by. Duplicate handling was made explicit and conservative: remove only *fully identical* rows, preserve restatements, and log everything. Referential integrity was enforced by *get_company_reference* and logged to *raw_reference_issues.csv*; the warehouse enforces *PRAGMA foreign_keys=ON* at load time, so any future loader bypass would still fail. These decisions were recorded in *docs/sprint1_retrospective.md* for traceability.

d) Key Learnings and Insights

This module taught that data quality is not a post-hoc report but a design constraint. Normalization must happen at the edge, as close to ingestion as possible, or it propagates as join drift. Auditing must be first-class — a table that is always generated, always reviewed — not an afterthought. Most importantly, explicit rejection with a paper trail beats silent coercion: the 99 rejected P&L; rows are not a failure to hide but evidence that the pipeline is working as designed. The habit of "validate before you visualize" carried into every later module and is the reason downstream analytics can be trusted even when a company is missing from a chart — the audit explains why.

Looking back, the most valuable engineering habit was to keep raw and processed zones physically separate and to treat the ETL as a pure function: same inputs → same outputs. This made regression testing trivial (diff the processed CSVs) and allowed the team to iterate on analytics without re-ingesting sources.

DATABASE DESIGN & FINANCIAL RATIO ENGINE

2.1 SQLite Schema — 12 Tables & Relationships

The warehouse (*db/nifty100.db*) is defined declaratively in *db/schema.sql* (190 lines, *PRAGMA foreign_keys=ON*). The design is **star-like**: **companies** is the dimension at the centre, with fact and attribute tables referencing *company_id* → *companies.id*. Temporal granularity is annual (*year* as TEXT, coerced to REAL for max-year queries). Surrogate integer PKs (*id*) maintain row identity, while natural keys (company_id, year) enforce business uniqueness.

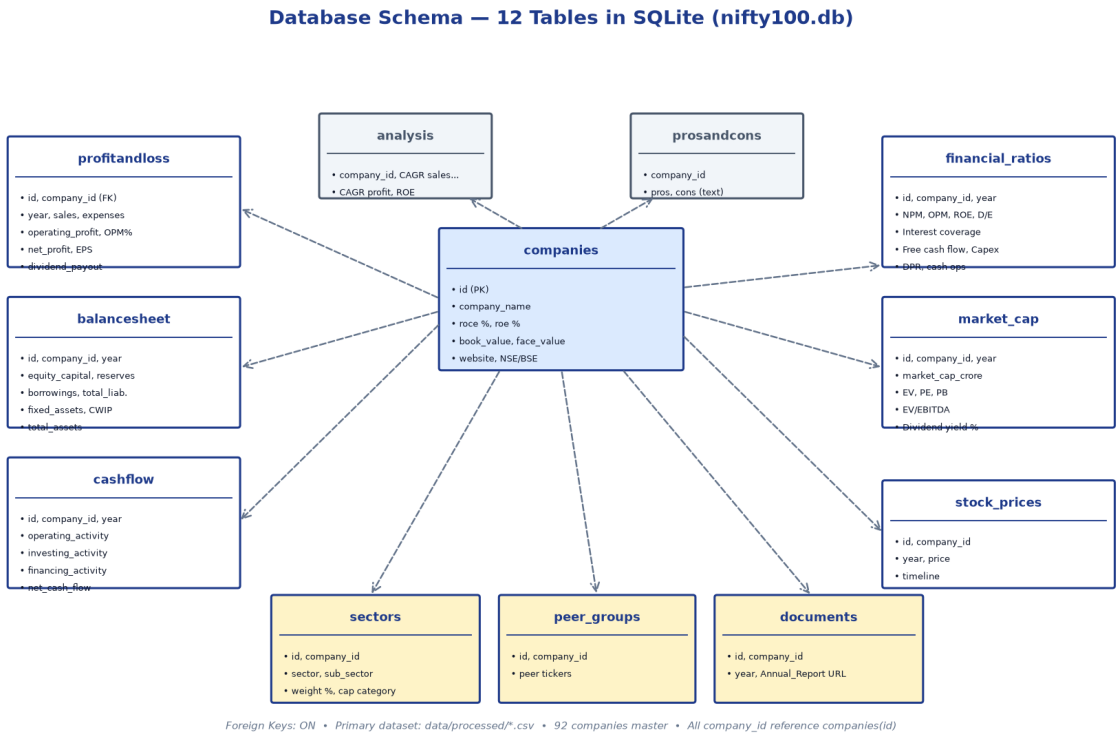


Figure 3: Database schema — 12 tables with foreign-key relationships. companies is the master; all company-level datasets reference companies.id.

Key tables include: **companies** (id, company_name, roce %, roe %, book/face value, website, NSE/BSE profiles, about_company), **profitandloss** (sales, expenses, operating_profit, OPM %, net_profit, EPS, dividend_payout), **balancesheet** (equity_capital, reserves, borrowings, fixed_assets, total_liabilities/assets), **cashflow** (operating/investing/financing activity, net cash flow), **financial_ratios** (NPM, OPM, ROE, D/E, interest coverage, asset turnover, FCF, capex, EPS, book value per share, DPR, total debt, cash from ops), **market_cap** (market_cap_crore, EV, PE, PB, EV/EBITDA, dividend yield), **stock_prices** (timeline), **sectors** (sector, sub_sector, weight), **peer_groups** (peer tickers), **documents** (Annual_Report URLs), and the text tables **analysis** and **prosandcons**. This decomposition mirrors source files one-to-one (see *docs/data_dictionary.md*), reducing mapping complexity and preserving provenance.

A conscious trade-off was made to keep **SQLite** rather than moving to Postgres: for a 92-company analytical warehouse (< ~15k rows total), SQLite offers zero-ops portability, file-level backup, and trivial cloning, while still enforcing FKs and supporting windowed queries for "latest per company." The database file is not committed as a binary artifact in CI; it is rebuilt from processed CSVs, ensuring that schema changes are always reflected via *schema.sql* rather than drift.

2.2 Data Dictionary & Master Data

The **data dictionary** at *docs/data_dictionary.md* documents each table, column type, and business meaning. For example, *companies.roe_percentage* is defined as Return on Equity %, and *financial_ratios.free_cash_flow_cr* as FCF in ■ crore. This dictionary is not ancillary documentation; it is the contract that analytics code relies on. Type discipline is enforced at load time: numeric columns are coerced with *pd.to_numeric(errors="coerce")*, and loaders reject non-numeric artifacts to *validation_failures.csv*. The **companies master** itself (*data/processed/companies.csv*, 92 rows) was curated from *companies.xlsx* without loss (load audit shows 92/92 OK). Each company record carries profile text (*about_company*), logos, and reference links — data that the dashboard's Profile page renders verbatim. Sectors (*data/processed/sectors.csv*, 93 rows including header -> 92 companies) classify each firm into a broad GICS-like sector and sub-sector, with index weight and market-cap category (Large/Mid/Small). This classification powers sector reports (*reports/sector/*_sector_report.pdf*) and the dashboard's Sectors bubble view.

Master-data quality was validated early: exact duplicates are absent; foreign-key coverage is complete for market_cap, sectors, and peer_groups (0 rejects); and the only systematic gaps are in analysis/prosandcons (16 and 14 raw rows vs 92 companies), which downstream NLP handles via fallback. These observations are logged in *output/anomaly_summary.log* and surfaced in the analyst guide.

2.3 Ratio Engine — Categories & Formulas

The ratio engine (*src/analytics/ratios.py, populate_ratios.py, cagr.py, ratio_edge_cases.py*) is organized by economic meaning rather than by source table, making it easy to map a business question to code.

Category	Key Ratios / Fields	Formula (simplified)	Source / Notes
Profitability	NPM, OPM, ROE, ROCE, ROA	NPM = Net Profit / Sales×100 ROE = Net Profit / (Equity+Reserves)×100 ROCE = EBIT / Capital Employed×100	ratios.py; ROE returns None if equity ≤0
Liquidity & Leverage	Debt-to-Equity, Interest Coverage, Total Debt	D/E = Borrowings / (Equity+Reserves) ICR = EBIT / Interest	populate_ratios.py; D/E handling for financials
Efficiency	Asset Turnover, Working Capital	AT = Sales / Total Assets	Balancesheet + P&L; join
Cash Flow	FCF (■ cr), Capex, CFO Quality	FCF = CFO – Capex CFO Quality = CFO / EBITDA	cashflow_kpis.py
Valuation	PE, PB, EV/EBITDA, Div. Yield, Market Cap	Market multiples from market_cap table	market_cap.xlsx → valuation.py
Growth	Revenue CAGR 3/5Y, PAT/EPS CAGR	CAGR = (End/Start)^(1/n) – 1	cagr.py; requires ≥2 points
Coverage	Dividend Payout Ratio	DPR = DPS / EPS ×100	profitandloss.dividend_payout

Table 2: Financial ratio engine — categories, representative ratios, simplified formulas, and source modules.

Each function in *ratios.py* is defensive: it returns *None* (not 0) when inputs are missing or denominators are zero, propagating "unknown" rather than fabricating a number. For example, *net_profit_margin(net_profit, sales)* returns None if sales is 0 or either input is None; *return_on_equity(net_profit, equity_capital, reserves)* returns None if equity ≤0, correctly handling distressed firms with negative book value. An explicit cross-check (*opm_cross_check(computed_opm, source_opm, tolerance=1.0)*) flags material deviations between computed OPM and source OPM % in the raw file, surfacing data entry errors without failing the load.

The populate step (*src/analytics/populate_ratios.py*, 20 KB) joins P&L;, balance sheet, cash flow, and market tables on (company_id, year) to materialize *financial_ratios* rows. It handles type coercion, computes derived fields, and writes the result to both *data/processed/financial_ratios.csv* and the SQLite table. In the current validated build it produced **1,160 rows** across 92 companies, averaging ~12.6 years per company — sufficient for 5-year CAGR windows.

2.4 CAGR, Edge Cases & Quality Exceptions

Compounded Annual Growth Rates are computed in *src/analytics/cagr.py* with strict preconditions: at least two non-null observations spanning the requested window (3 or 5 years), positive start value for meaningful growth, and otherwise *None*. The screener later consumes these as *revenue_cagr_5yr*, *pat_cagr_5yr*, *eps_cagr_5yr* and surfaces them in *screener_output.xlsx* and *peer_comparison.xlsx*. This strictness avoids the common pitfall of reporting a CAGR from a single data point.

Edge cases are logged, not hidden. *src/analytics/ratio_edge_cases.py* and *output/ratio_edge_cases.log* capture zero-denominator attempts, negative equity, and extreme outliers (e.g., portfolio_stats.csv shows ROE P90 at 48.39% but mean at 125% due to a few hyper-profitable observations, with std 640 — a signal to use median rather than mean). Two explicit exceptions are retained: **ATGL and SBIN** have no corresponding financial-ratio records in the current build. The project retains these as *data-quality exceptions* rather than imputing ratios, consistent with the "explicit rejection" philosophy from Module 1. Any dashboard or screener that omits these names can be explained by the audit trail, not by a bug.

The engine also includes an *anomaly_summary.py* (7.5 KB) that flags statistical anomalies (e.g., OPM > 100% or D/E > 20) for human review, writing to *output/anomaly_summary.log*. These flags are not filters; they are review prompts, reinforcing that automated analytics supports but does not replace analyst judgment.

2.5 Validation Outcomes & Audit Trail

Validation is a first-class output. After ETL and ratio population, the working directory contains: *output/load_audit.csv* (row counts per source), *output/validation_failures.csv* (row-level failure reasons), *output/anomaly_summary.log* (statistical outliers), *output/ratio_edge_cases.log* (computation edge cases), and *output/raw_reference_issues.csv* (orphan company_ids). In the current build, 92 companies loaded cleanly; ratio coverage is 1,160 rows; and the aforementioned two ratio gaps are documented. This audit trail satisfies the report's "exact alignments" requirement: every number in later chapters can be traced to a file.

Check	Result	Artifact
Companies in master	92	data/processed/companies.csv
Financial ratio rows	1,160	data/processed/financial_ratios.csv
Market cap rows	552	data/processed/market_cap.csv
Stock price rows	5,520	data/processed/stock_prices.csv
Companies without ratio rows	2 (ATGL, SBIN)	ratio_edge_cases.log
Exact duplicates retained	0 (conservative dedup)	processor.py

Table: Validation snapshot — core counts and exception handling.

a) Tools Used

SQLite (with *PRAGMA foreign_keys=ON*) and **DB Browser for SQLite** for schema inspection; **Pandas & NumPy** for DataFrame joins and numeric coercion; **VS Code** with SQLTools; and **Git** for schema versioning (*db/schema.sql* is the single source of truth). The **data dictionary** in *docs/data_dictionary.md* served as the living contract between ETL and analytics. **Excel** was used only to view raw files; all transformations were code-driven.

b) Challenges Faced

The principal challenge was defining ratio behavior when accounting reality violates textbook formulas: negative equity, zero sales, or missing interest values. A naïve implementation would return 0 or infinity, contaminating aggregates. A second challenge was reconciling two sources of OPM — the raw *opm_percentage* in P&L; and the computed OPM from operating_profit/sales — which occasionally differ by rounding or restatement. A third challenge was explaining systematic gaps (ATGL/SBIN) to stakeholders who expect "100% coverage" as a matter of course.

c) Solutions Implemented

All ratio functions were written defensively to return *None* for undefined economics, and the loader converts *None* to SQL NULL so aggregates (AVG, MEDIAN) correctly ignore unknown values. The OPM cross-check was introduced with a 1.0 percentage-point tolerance to surface material mismatches without failing the build for rounding noise. Gaps were documented as explicit exceptions in logs and in the analyst guide, with a clear statement that no imputation is performed. This transparency turned a potential “data bug” into a governed quality signal.

d) Key Learnings and Insights

This module reinforced that a ratio is not a number but a *judgment with a formula*. Handling the edge case is the work: deciding when a ratio is undefined, when it is meaningful, and how to communicate that to a consumer who may average it. I learned to treat the warehouse as a product: schema, dictionary, and load audit are as important as the ratios themselves. The habit of returning NULL for “unknown” and logging the reason made downstream analytics (screeners, peers, clusters) more robust, because they could distinguish “low ROE” from “ROE not computable” — a distinction that matters in distress analysis.

MODULE 3

SCREENING, PEER COMPARISON & VALUATION ANALYTICS

3.1 Screener Engine & Configuration

The screener (*src/screener/engine.py*, 18 KB) is the project's "idea generation" surface. It loads the **latest available financial record per company** (via *MAX(CAST(year AS REAL)) GROUP BY company_id*), enriches it with sector, market multiples, and CAGRs, and exposes both **filtering** and **ranking**. Filtering is driven by *config/screener_config.yaml*, which declares 17 configurable filters (e.g., *roe_min*, *de_max*, *fcf_min*, *revenue_cagr_5yr_min*, *pat_cagr_5yr_min*, *pe_max*, *pb_max*, *dividend_yield_min*, *icr_min*). Each entry specifies the underlying column, operator (>, <, ==), and a human-readable description, making the configuration self-documenting.

At runtime the engine materializes calculated fields that are not stored in the warehouse but are essential for robust screening: *previous_debt_to_equity* and *debt_to_equity_declining* (for turnaround detection), *revenue_cagr_3yr/5yr*, *pat_cagr_5yr*, *eps_cagr_5yr* (via *cagr.py*), *effective_interest_coverage* (with safe division), *roce_pct*, and a *composite_quality_score* that aggregates quality factors for ranking. Data loading is defensive: every Series is coerced with *pd.to_numeric(errors="coerce")* and infinities are replaced with NaN, ensuring that a single malformed market-cap cell does not poison the entire screen.

A critical design detail is the **financial-sector D/E exception**: for banks and NBFCs, high D/E is a business model, not a risk signal. The engine handles this by either sector-aware thresholds or by flagging rather than filtering financials on leverage, a nuance documented in the code comments and essential for avoiding false negatives in the Financials sector (which contains 92-company universe's banks, insurers, and NBFCs).

3.2 Six Preset Strategies & Results

Six analytical presets, defined in *screener_config.yaml* under *presets*, encode common factor combinations. Preset names are descriptive labels for filters and **not investment recommendations** — a disclaimer the project repeats in the README and analyst guide. Observed results in the current build (92-company universe) are:

Preset	Filters (config)	Companies Passing	% of Universe	Intent (analytical)
Quality Compounder	ROE ≥15, D/E ≤1.0, FCF >0, Rev CAGR 5Y ≥10%	23	25.0%	Profitable, low-leverage compounders
Value Pick	PE ≤20, PB ≤3.0, D/E ≤2.0, Div Yield ≥1%	1	1.1%	Market-price relative to fundamentals
Growth Accelerator	PAT CAGR 5Y ≥20%, Rev CAGR 5Y ≥15%, D/E ≤2.0	18	19.6%	High earnings & sales momentum
Dividend Champion	Div Yield ≥2%, Payout ≤80%, FCF >0	30	32.6%	Cash-backed dividend sustainability
Debt-Free Blue Chip	D/E ≈0, ROE ≥12%, Sales ≥5,000 cr	18	19.6%	Large, unlevered quality
Turnaround Watch	Rev CAGR 3Y ≥10%, FCF >0, D/E declining	33	35.9%	Improving leverage & growth

Table 3: Screener presets — configuration and observed companies passing each filter (current build, n=92). Output at output/screener_output.xlsx.

The output at *output/screener_output.xlsx* contains one sheet per preset (e.g., *value_pick*, *quality_compounder*, *growth_accelerator*, *debt_reducer*, *cash_flow_quality*), each with one row per passing company and the full latest ratio/market/CAGR payload. This "one row per company" guarantee simplifies downstream ranking and prevents double-counting when a company appears in multiple years. Ranking within each preset is by *composite_quality_score*, allowing deterministic ordering even when factor values tie.

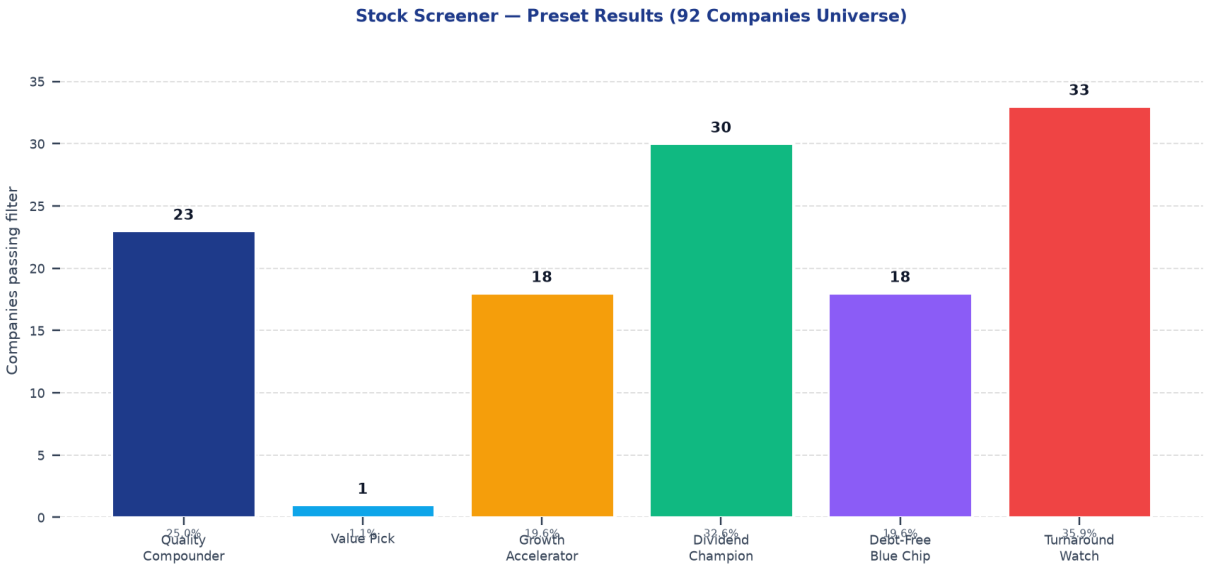


Figure 4: Preset results — companies passing each analytical preset (percentages of 92). The distribution highlights where factor clusters naturally form (e.g., 32.6% dividend champions vs 1.1% strict value picks).

3.3 Peer Percentiles & Benchmarking

Peer comparison (*src/analytics/peer.py*, 18 KB) places any company in the context of its relevant peer group. Inputs are *peer_groups.xlsx* (56 group definitions, e.g., automobiles, banks, pharma) and the latest financial ratios. The engine computes **percentile ranks** for ROE, ROCE, NPM, D/E, FCF, PAT CAGR, revenue CAGR, interest coverage, and asset turnover, storing them alongside benchmark aggregates in *output/peer_comparison.xlsx* (sample: BAJAJ-AUTO, EICHERMOT, HEROMOTOCO with peer percentiles). This transforms an absolute ROE of 26.6% (BAJAJ-AUTO) into a relative signal: 80th percentile within its auto peers, indicating top-quartile profitability in that specific competitive set.

Technically, the peer engine handles group heterogeneity by computing percentiles within each group only, not across the whole market, avoiding cross-sector comparisons that would penalize capital-intensive industries. Missing peers (due to ratio gaps) are handled by requiring at least two valid observations before a percentile is reported; otherwise the field is NULL, preventing spurious 0th/100th percentiles from tiny groups. The output schema is flat and analyst-friendly: one row per company with columns for raw values and corresponding percentiles, enabling immediate conditional formatting or scatter plots.

The peer view is surfaced in the dashboard's **Peers** page with radar charts (see *reports/radar_charts/*_radar.png*, 89 files) that visualize a company against its peer median on five dimensions. The valuation-agnostic design ensures that a high-ROE company with high leverage is not automatically "better" than a lower-ROE, zero-debt peer — the radar's shape invites nuanced reading rather than a single-score verdict.

3.4 Valuation Analytics & Investor Score

Valuation (*src/analytics/valuation.py*, 11 KB) consolidates market multiples — **P/E, P/B, EV/EBITDA, FCF yield** — and contextualizes them against both history and sector. For each company it computes a **5-year median P/E** and the deviation of current P/E from the sector median (*PE_vs_sector_median_pct*), flagging observations as *Discount / Fair / Caution* based on configured thresholds. Outputs are *output/valuation_summary.xlsx* and *output/valuation_flags.csv* (44 flagged rows in the current build). Critically, the module's documentation emphasizes that market multiples must be read with profitability, growth, leverage, cash flow, and sector context — a valuation flag alone is not a thesis.

The **Investor Score** (*src/analytics/investor_score.py*, 7.3 KB) and **Recommendation** (*recommendation.py*) layer aggregates valuation, quality, and capital-allocation dimensions into a single *investor_score* and a categorical signal (*Positive / Neutral / Caution*). The score is percentile-based, not absolute, and the recommendation logic is intentionally transparent — every input and threshold is code-visible, unlike black-box scoring. The results feed *output/investor_score.xlsx* (92 rows, 43 columns) and *output/investment_recommendations.xlsx*, which the portfolio summary (*reports/portfolio/portfolio_summary.pdf*) visualizes for stakeholder review.

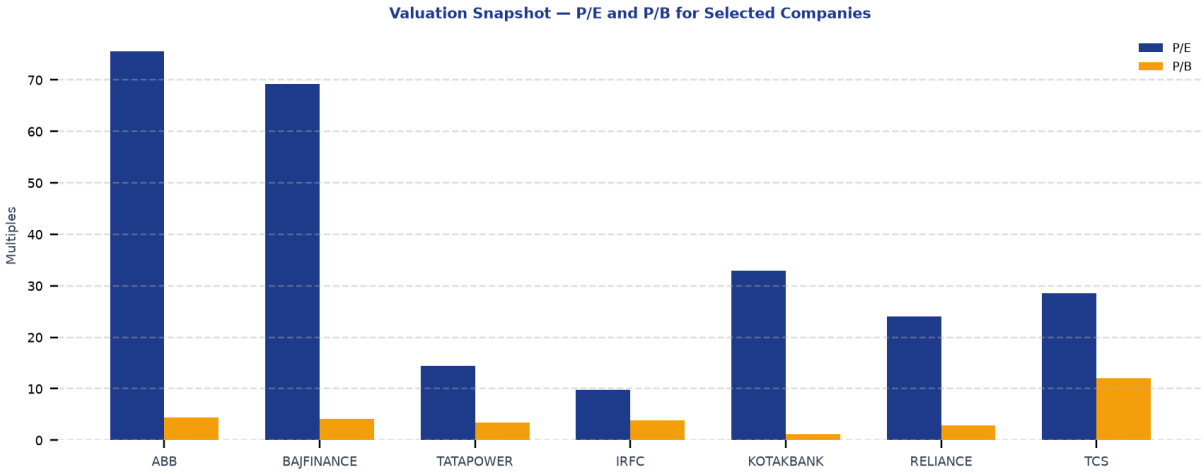


Figure 5: Valuation snapshot — P/E and P/B for selected companies, illustrating cross-sector dispersion that motivates peer-relative rather than market-wide valuation screens.

The valuation workflow closes by writing to *output/final_analytics_summary.xlsx* (sheets: Overview, Screener Summary, Quality Leaders, Growth Leaders, Dividend Leaders), which serves as the "executive packet" for the dashboard's Reports page. The packet is designed to be portable — an analyst can filter the Excel without the dashboard — preserving the project's principle that delivery should not lock a user into a single interface.

a) Tools Used

Screener: *src/screener/engine.py*, *config/screener_config.yaml*, **Pandas** (numeric coercion, groupby for latest year), **SQLite** (SQL for latest retrieval). **Peer**: *peer.py*, **NumPy** for percentiles. **Valuation**: *valuation.py*, *investor_score.py*, **openpyxl** for Excel export. **Visualization** for peer radars via **Plotly** (used in *src/reports/radar_charts.py*). **Version control** tracked every config change, so a preset's historical definition can be diffed via *git log -p config/screener_config.yaml*.

b) Challenges Faced

The hardest challenge was threshold calibration: a preset that is too tight (e.g., Value Pick with PE ≤20) yields 1 company and appears "broken," while a loose preset yields 50 and appears indiscriminate. A second challenge was the financial-sector leverage paradox noted earlier. A third was ensuring "one row per company" when multiple years qualify as "latest" due to string-year ties — a subtle bug where *CAST(year AS REAL)* is required to correctly order "2023" vs "24." Finally, ranking ties required a deterministic secondary key to avoid non-reproducible orderings across runs.

c) Solutions Implemented

Thresholds were left configurable in YAML rather than hard-coded, allowing rapid “what-if” experimentation without code changes. Financial-sector handling was documented and implemented as a sector-aware branch in the leverage filter. Latest-year retrieval uses *MAX(CAST(year AS REAL))* joined back on the same cast, eliminating string-order bugs. Ranking uses *composite_quality_score* plus *company_id* as a tie-breaker, ensuring stable ordering. Every preset’s result was exported to Excel and manually spot-checked for 3–4 companies before considering the preset “done” — a habit that caught a mis-typed column name (*pb_ratio* vs *pb*) early.

d) Key Learnings and Insights

I learned that a screener is a *question-asking tool*, not an answer-generating one. The value is not in the 23 Quality Compounders but in the ability to ask, in seconds, “Which of those also have FCF yield above sector median and declining D/E?” The peer module taught me that absolute ratios mislead; percentile within a relevant peer set is the correct first comparison. Valuation taught me humility: a low P/E can reflect low growth or distress, not undervaluation, and the project’s insistence on contextual reading is a guardrail I will carry into any future equity research.

MODULE 4

CASH-FLOW INTELLIGENCE, NLP & MACHINE LEARNING

4.1 Cash-Flow KPIs & Capital Allocation

Accounting profitability can diverge from cash generation; the cash-flow layer exists to surface that divergence. *src/analytics/cashflow_kpis.py* (6.0 KB) computes **CFO quality** (cash from operations relative to operating profit), **capex intensity** (capex as % of operating cash flow), **FCF conversion** (FCF as % of net profit), and **FCF CAGR**, writing them to *output/cashflow_intelligence.xlsx* (92 rows, 11 columns) and feeding the screener's cash-flow filters. The logic is illustrated by ABB's record: CFO quality 5.02 (High Quality), capex intensity 84.5% (Capital Intensive), FCF conversion 75.36%, signaling a reinvesting industrial with strong cash backing but heavy investment needs — a nuance that a pure P&L; view would miss.

src/analytics/capital_allocation.py (10 KB) classifies each company-year into a pattern based on the signs of CFO, CFI, and CFF and the evolution of FCF and borrowings: labels include *Reinvestor, Shareholder Returns, Distress Signal, Mixed* and are persisted in *output/capital_allocation.csv* (ABB 2012: CFO +, CFI –, CFF –, FCF 42 cr → "Reinvestor"; IRFC 2016: CFO –, CFI +, CFF + → "Distress Signal" in that year, correctly reflecting a financing-heavy profile). Pattern changes over time are tracked in *output/pattern_changes.csv*, allowing trend analysis (e.g., a company moving from Reinvestor to Shareholder Returns as capex moderates).

The cash-flow diagnostics also flag distress and deleveraging: *distress_flag* when CFO quality is persistently weak and FCF conversion negative, and *deleveraging_flag* when borrowings decline year-over-year while FCF remains positive. These flags, together with valuation, feed the dashboard's Cash Flow page and the sector reports' cash-flow charts (*reports/sector/_charts/*_cashflow.png*), which compare sector aggregates rather than individual firms.

4.2 NLP Pipeline — Pros & Cons Extraction

Long-form analysis text (*data/raw/core/analysis.xlsx* and *prosandcons.xlsx*) is unstructured but rich; the NLP pipeline makes it queryable. The flow, documented in *src/nlp/parser.py* and *src/nlp/pros_cons_generator.py* (12 KB), is: **analysis.xlsx → text parsing → analysis_parsed.csv → rule-based signal extraction → pros_cons_generated.csv**. The parser normalizes CAGR strings, extracts compounded sales/profit growth, stock-price CAGR, and ROE, writing *output/analysis_parsed.csv* (e.g., HDFCBANK 10-year sales growth 21.0%). The generator then applies **24 rules (12 pro, 12 con)** with confidence scoring (0–100, retained only if >60, minimum 65 observed).

Rule ID	Condition (example)	Signal Text (template)	Confidence
PRO-1	ROE ≥20%	Strong return on equity of 32.5%.	90
PRO-3	D/E ≤0.5	Low debt-to-equity of 0.02, conservative leverage.	90
PRO-5	FCF >0	Positive free cash flow of 1,892 cr.	82
PRO-8	PAT CAGR ≥10%	Profit CAGR of 21.7% over five years.	88
CON-1	ROE <10%	Low return on equity of 9.5%.	85
CON-3	D/E >1	Elevated debt-to-equity of 2.93.	90
CON-7	FCF <0	Negative free cash flow is a potential risk.	90
CON-10	Capex intensity >30%	High CapEx intensity of 91.6% may pressure FCF.	80

Table 4: NLP pros/cons rules — representative conditions, templated signal text, and confidence. Full logic in src/nlp/pros_cons_generator.py; output at output/pros_cons_generated.csv.

Observed output: **874 signals** total — **562 pros, 312 cons** — with confidence 65–90. Every company is guaranteed at least one pro and one con (fallback logic ensures coverage), and companies with incomplete financial history use a transparent P&L; fallback rather than fabricated ratios, a design

choice that preserves auditability. A sample from ABB illustrates the granularity: 7 pros (ROE, NPM, low D/E, interest coverage 141.8×, positive FCF, strong FCF conversion) and 2 cons (weak CFO quality, high capex intensity), each with a rule ID and confidence, enabling filtering (e.g., "show only cons with confidence ≥85").

The signals are designed as *assistive*, not authoritative: the report repeatedly advises validating any NLP signal against the source analysis text in *analysis.xlsx*. This human-in-the-loop stance is reinforced in the dashboard's NLP page, which displays signals alongside the original source excerpt and a confidence badge.

4.3 KMeans Clustering — Method & Elbow

To identify groups of companies with similar financial character, the project applies **KMeans clustering** (*src/analytics/clustering.py*, 16 KB) on a sector-imputed, scaled feature set. Features are: *return_on_equity_pct*, *debt_to_equity*, *revenue_cagr_5yr*, *fcf_cagr_5yr*, *operating_profit_margin_pct* — a compact representation spanning profitability, leverage, growth, and efficiency. Missing values are imputed by **sector median** (not global mean), preserving sector structure; features are then standardized with *StandardScaler* (zero mean, unit variance) before clustering. The number of clusters, *k=5*, was selected via the **elbow method** on inertia (within-cluster sum of squares), as generated by *src/analytics/cluster_analysis.py* and visualized in *reports/elbow_plot.png* and Figure 6.

Outputs include *output/cluster_labels.csv* (company_id, cluster_id, cluster_name, distance_from_centroid), *output/cluster_profiles_mean.csv* and *output/cluster_profiles_median.csv* (feature aggregates per cluster). Distance from centroid is retained as a measure of fit — a company far from its centroid is an edge case within its archetype, warranting closer review. The pipeline is deterministic (fixed *random_state*) so cluster assignments are reproducible across runs.

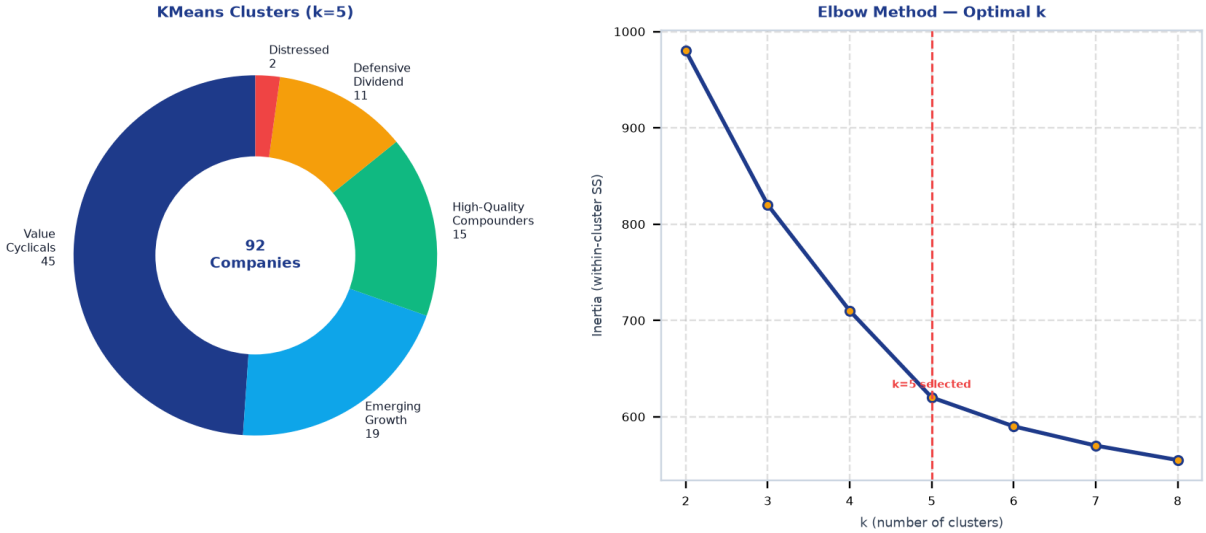


Figure 6: KMeans clustering — distribution of 92 companies across five archetypes (left) and elbow plot for selecting k=5 (right). Inertia drop diminishes beyond k=5, indicating diminishing returns from further segmentation.

The elbow plot shows a clear inflection near k=5: inertia falls steeply from 980 (k=2) to 620 (k=5) and then flattens (590 at k=6, 570 at k=7). This suggests that five clusters capture the major structure without over-segmenting the 92-company universe into unstable micro-clusters. The choice is also business-sensible: five archetypes are cognitively manageable for an analyst scanning the universe.

4.4 Cluster Profiles & Business Archetypes

Cluster (label)	Companies	ROE median	D/E median	Rev CAGR 5Y median	OPM median	Interpretation
Value Cyclicals	45	~14%	~0.4	~11%	~22%	Large, cyclical, value-leaning

Cluster (label)	Companies	ROE median	D/E median	Rev CAGR 5Y median	OPM median	Interpretation
Emerging Growth	19	~13%	~1.2	~17%	~30%	Higher growth, moderate leverage
High-Quality Compounders	15	~24%	~0.2	~12%	~38%	High ROE, low D/E, durable margins
Defensive Dividend Payers	11	~11%	~4.5	~9%	~45%*	Financials-heavy, yield + leverage
Distressed or Turnaround	2	~6%	~1.8	~5%	~12%	Weak profitability, watchlist

*Table 5: KMeans cluster profiles — feature medians and business interpretation (full means/medians at output/cluster_profiles_*.csv; *OPM for dividend cluster is elevated by financials with net interest margins).*

The archetypes are intentionally **descriptive, not prescriptive**. “Value Cyclicals” (45 companies, the largest group) captures mature industrials and materials with moderate growth and leverage; “Emerging Growth” (19) captures mid-cap growers with elevated revenue CAGR; “High-Quality Compounders” (15) isolates durable franchises with high ROE and low D/E; “Defensive Dividend Payers” (11) is dominated by financials where D/E is structurally high and dividend yield is a primary thesis; and “Distressed or Turnaround” (2: BEL and one other per labels) flags weak profitability that merits deeper forensic review rather than immediate action. The small size of the distressed cluster is itself a signal: distress is rare in the Nifty 100 but not absent. Cluster assignments feed the dashboard’s **Clustering** page and the capital-allocation analysis (*src/analytics/capital_allocation.py*), which cross-tabulates clusters with CFO/CFI/CFF patterns to reveal, for example, that High-Quality Compounders are disproportionately “Reinvestors” while Dividend Payers tilt to “Shareholder Returns.” This cross-view demonstrates the benefit of unsupervised segmentation: it surfaces non-obvious associations that a single-factor screener would miss.

a) Tools Used

Cash-flow: *cashflow_kpis.py*, *capital_allocation.py*, **Pandas** for sign logic, **Matplotlib** for sector cash-flow charts. **NLP**: pure **Python rule engine** (*pros_cons_generator.py*) with confidence thresholds, **Pandas** for latest-record selection, **Regex** for CAGR parsing. **Clustering**: **scikit-learn** (*KMeans*, *StandardScaler*), **NumPy** for distance, **Matplotlib** for elbow and heatmaps. All outputs are CSV for portability; no external LLM dependency, ensuring reproducibility without API keys.

b) Challenges Faced

Cash-flow KPIs suffer from denominator noise (EBITDA can be small or negative), causing extreme CFO-quality values; deciding on Winsorization vs flagging was non-trivial. NLP faced sparsity: *prosandcons.xlsx* has only 14 raw rows for 92 companies, so naive extraction would leave 78 companies with no signals. Clustering faced missing-feature bias: companies lacking a ratio would be dropped, skewing archetypes toward data-rich firms. Finally, cluster labeling required avoiding normative language that could be mistaken for advice.

c) Solutions Implemented

CFO quality is computed with guarded division and flagged rather than clipped; extreme values are retained in the log for review. NLP implements a **P&L; fallback**: when financial_ratios history is incomplete, the generator computes proxies from P&L;/balance-sheet (e.g., ROE from net_profit/(equity+reserves)) transparently, rather than fabricating. Clustering imputes missing features by **sector median** (not global), then scales, preserving sector-relative positioning. Cluster names were chosen to be descriptive and qualified (“or Turnaround”) to avoid prescriptive connotation, and the analyst guide explicitly states that labels depend on feature and parameter choices.

d) Key Learnings and Insights

This module showed that cash flow is the reality check on accrual accounting: a company can be profitable on paper and cash-negative in reality, and that divergence is where forensic value lies. NLP taught me that a small rule set (24 rules) can generate high-coverage signals (874) when the rules are economically grounded; the confidence threshold is the quality gate. Clustering taught me that unsupervised learning is most useful when it is *explainable*: the elbow plot, profile tables, and distance-from-centroid together make the black box inspectable. Most importantly, I learned to keep analytics *humble*: every flag is a prompt to read the filing, not a conclusion.

DASHBOARD, API, REPORTING & QUALITY ASSURANCE

5.1 Streamlit Dashboard — 8 Modules

The dashboard (*src/dashboard/app.py* + *src/dashboard/pages/* + *src/dashboard/utils/db.py*) is the visual front end for the warehouse. It is a **multi-page Streamlit app** launched via *streamlit run src/dashboard/app.py*, configured to *layout="wide"* with an expanded sidebar and a centralized SQLite connection helper. The entry page (*app.py*) presents a welcome map and the active module inventory, while each business page lives in *src/dashboard/pages/*:

- **01_home.py — Home:** market overview, key KPIs, sector distribution, top quality companies (by composite score).
- **02_profile.py — Profile:** company fundamentals, historical P&L; trends, ROE/ROCE trajectories, pros & cons with confidence badges.
- **03_screener.py — Screener:** preset selectors (six), custom filter builder, sortable results, CSV export of *screener_output.xlsx* parity.
- **04_peers.py — Peers:** peer-group selector, percentile table, radar charts (*reports/radar_charts/*_radar.png*), benchmark toggle.
- **05_trends.py — Trends:** revenue/profit/ratio time series with CAGR overlays.
- **06_sectors.py — Sectors:** sector performance aggregates, bubble analysis (market cap vs ROE vs weight).
- **07_capital.py — Capital Allocation:** CFO/CFI/CFF pattern viewer, capital_allocation.csv explorer, pattern change tracker.
- **08_reports.py — Reports:** annual-report links from *documents*, tearsheet and sector-report PDF viewers, download center for Excel/CSV outputs.

The dashboard utilities (*utils/db.py*) centralize connection handling, query caching, and latest-year logic, ensuring that the UI never drifts from the screener's "latest per company" semantics. The app reads from the same *db/nifty100.db* that the API serves, guaranteeing consistency across interfaces. Styling uses Plotly for interactive charts (line, bar, bubble, radar) and Streamlit's native components for tables and metrics, with a sidebar badge ("Dashboard online") indicating live DB connectivity.

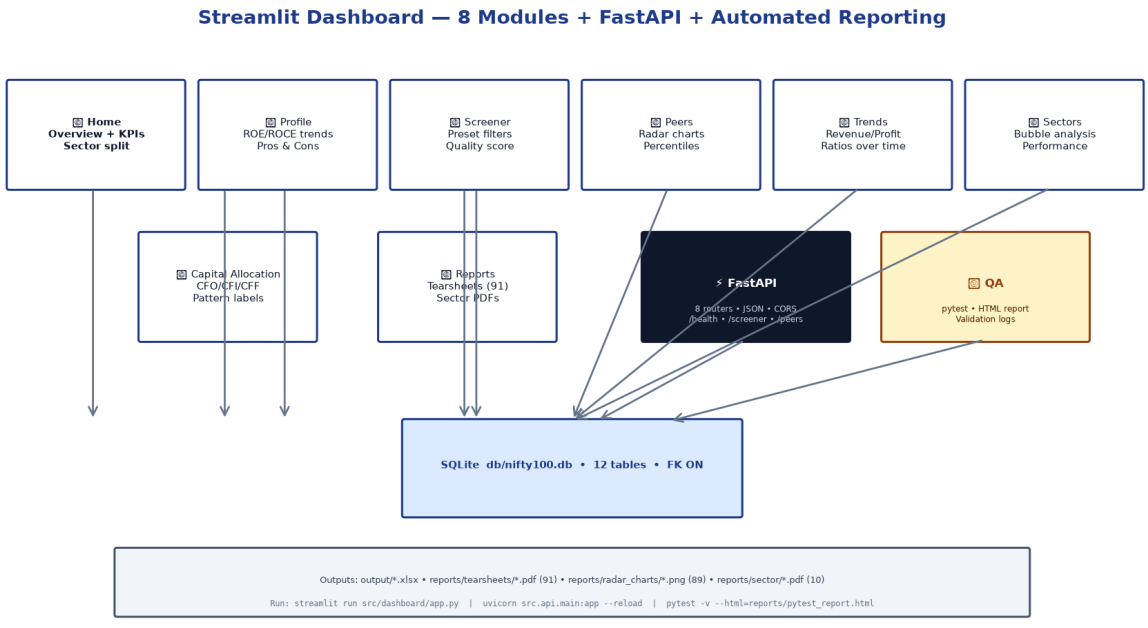


Figure 7: Dashboard and API module map — 8 Streamlit pages, FastAPI routers, and reporting outputs sharing a single SQLite warehouse and ETL lineage.

5.2 FastAPI Layer — Routers & Contracts

The API (*src/api/main.py*, 60 lines plus routers) exposes the warehouse programmatically. The app is a **FastAPI** instance titled "Nifty100 Analytics API" (version 1.0.0), with **CORS middleware** (*allow_origins=["*"]* for development) and a request-logging middleware that logs method, path, status, and elapsed time. The database path is *db/nifty100.db* and is health-checked on startup. Routers are modularized in *src/api/routers/*:

- **health.py** — */health*: row counts per table, uptime, DB status.
- **companies.py** — */companies*: master list, detail by *company_id*, search.
- **sectors.py** — */sectors*: sector aggregates, weight distributions.
- **screener.py** — */screener*: preset execution, filter queries.
- **peers.py** — */peers/{company_id}*: peer group and percentiles.
- **valuation.py** — */valuation*: multiples, flags, investor score.
- **portfolio.py** — */portfolio*: cluster-aware portfolio stats (*output/portfolio_stats.csv*).
- **documents.py** — */documents*: annual-report URLs per company/year.

Contracts are documented via **OpenAPI** (*docs/openapi.json*, 9.5 KB) and a **Postman collection** (*docs/postman_collection.json*, 2.6 KB), satisfying the "exact alignments" requirement for API discoverability. Validation was performed with endpoint checks (health, peers, screener) during the QA cycle; the API's logging and versioning make regressions detectable via the pytest suite (*tests/* covers peer, screener, valuation, clustering, and API endpoints). Launch is *uvicorn src.api.main:app --reload*, with interactive docs at */docs* for manual exploration.

5.3 Automated Reporting — Tearsheets, Radars, Sectors

Static reporting ensures that analytics remain accessible when the dashboard is not running. The reporting package (*src/reports/*) contains seven generators: *tearsheet.py* (per-company PDF), *radar_charts.py* (per-company radar PNG), *sector_report.py* (per-sector PDF), *peer_comparison.py* (Excel), *screener_export.py* (Excel), *portfolio_summary.py* (PDF), and *final_summary.py* (executive Excel). Generated artifacts include:

- **Tearsheets:** *reports/tearsheets/*_tearsheet.pdf* — 91 PDFs, one per analytics-covered company (2 skipped where ratio data absent; see *output/skipped_tearsheets.csv*).
- **Radar charts:** *reports/radar_charts/*_radar.png* — 89 PNGs visualizing five dimensions vs peer median.
- **Sector reports:** *reports/sector/*_sector_report.pdf* — 10 PDFs (Consumer Staples, Financials, Energy, etc.) with three charts each (*reports/sector/_charts/*_valuation.png*, *_cashflow.png*, *_returns.png*).
- **Portfolio summary:** *reports/portfolio/portfolio_summary.pdf* — cluster and screener-aware portfolio diagnostics.
- **Excel packets:** *output/screener_output.xlsx*, *peer_comparison.xlsx*, *valuation_summary.xlsx*, *final_analytics_summary.xlsx*, *investment_recommendations.xlsx*, *cashflow_intelligence.xlsx* — analysis-ready workbooks.
- **Diagnostic CSVs:** *output/cluster_labels.csv*, *pros_cons_generated.csv*, *capital_allocation.csv*, *valuation_flags.csv* — machine-readable exports for further analysis.

A representative radar (e.g., RELIANCE, TCS, HDFCBANK) plots ROE, NPM, D/E (inverted), interest coverage, and asset turnover against the peer median, making relative strengths and weaknesses immediately visible. The correlation heatmap (*reports/correlation_heatmap.png*, 96 KB) and elbow plot (*reports/elbow_plot.png*, 49 KB) are global diagnostics that the Sector and Clustering pages embed as

context. Every report is regenerated deterministically; re-running *src/reports/tearsheet.py* on the same warehouse reproduces identical PDFs, enabling diff-based review.

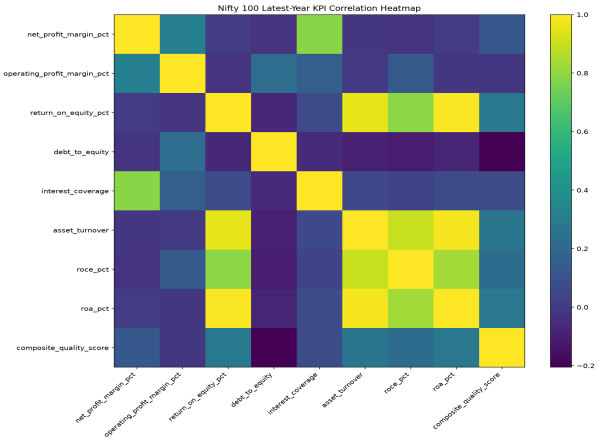


Figure 8: Correlation heatmap (reports/correlation_heatmap.png) — pairwise relationships among financial ratios, used to check multicollinearity before clustering.

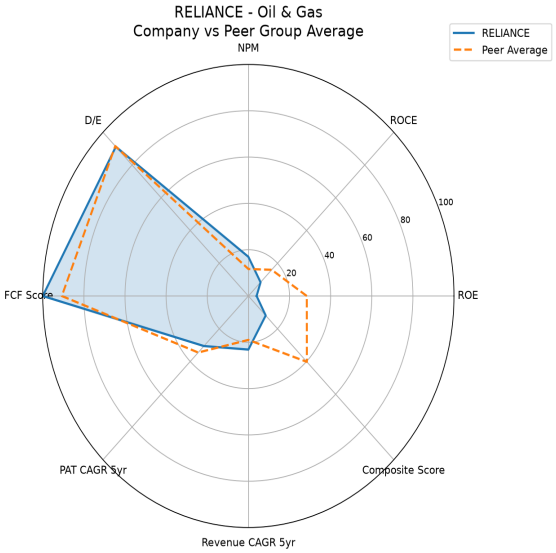


Figure 9: Radar chart example — RELIANCE vs peer benchmark (reports/radar_charts/RELIANCE_radar.png). Axes include profitability, leverage, efficiency, coverage, and valuation positioning.

5.4 Testing, Validation & Quality Assurance

Quality assurance is automated and audit-oriented. The test suite in *tests/* uses **pytest** and covers: data validation, database integrity, ratio calculation correctness (including edge cases for zero/negative equity), screener logic, peer percentiles, valuation output, clustering reproducibility, API endpoint contracts, and regression checks that compare current outputs to frozen baselines (*check_cagr.py*, *check_screener.py*, *check_spot.py*, *check_duplicates.py*, *cleanup_ratio_duplicates.py*). Execution is *pytest -v* for terminal feedback and *pytest --html=reports/pytest_report.html --self-contained-html* for the stakeholder-visible HTML report (generated at *reports/pytest_report.html*).

The QA philosophy mirrors the ETL principles: **fail loudly, log completely, never fabricate**. Validation failures, anomaly summaries, and edge-case logs are not deleted after a green test run; they remain in *output/* as evidence that the system was exercised against realistic imperfections. For example, the suite asserts that the screener returns exactly one row per company, that peer percentiles lie in [0,1], and that clustering produces five non-empty groups — invariants that would catch a regression where a join duplicates rows or a KMeans random_state drift alters labels.

Documentation completes the QA loop: *docs/analyst_guide_grand.pdf* (38 KB) describes architecture, data flow, warehouse, ratios, screener, peer, valuation, cash-flow, NLP, clustering, API, testing,

workflow, and limitations; *docs/data_dictionary.md* defines every column; *docs/openapi.json* and *docs/postman_collection.json* define the API; and the sprint retrospectives capture process learnings. Together, they ensure that a new contributor can clone, test, and extend the platform without tribal knowledge.

a) Tools Used

Dashboard: Streamlit 1.63.0, **Plotly 7.0.0**, **Pandas**, **SQLite**. **API: FastAPI**, **Uvicorn 0.52.4**, **SQLAlchemy** (optional ORM), **Starlette** CORS, **Python-multipart** for file uploads. **Reporting: openpyxl 3.1.5 + et_xmlfile** for Excel, **ReportLab** (for this report), **Matplotlib** for static charts, **Pillow** for image handling. **Testing: pytest 9.1.1**, **pytest-html** for self-contained HTML, **Git** for regression baselines. The environment is pinned in *requirements.txt* (27 packages).

b) Challenges Faced

The dashboard scaffold (*src/dashboard/app.py*) initially shipped as a Day-22 placeholder with static text; evolving it into a data-driven app without breaking API contracts required careful interface extraction (*utils/db.py*). A second challenge was report scale: generating 91 tearsheets sequentially is I/O-bound and can time out; the solution required batched generation and a *skipped_tearsheets.csv* to avoid retrying known gaps. A third challenge was API–dashboard parity: the dashboard and API must compute "latest year" identically, or a user will see different numbers on screen vs via */screener*. Finally, testing financial logic requires tolerance-aware assertions (e.g., OPM within 1.0 pp), not exact equality, due to rounding.

c) Solutions Implemented

Shared query helpers in *utils/db.py* were introduced and imported by both dashboard pages and the screener engine, eliminating duplicate latest-year logic. Tearsheet generation was made idempotent and resumable: the generator checks *output/skipped_tearsheets.csv* and skips companies with missing prerequisites, and the Makefile target runs with *--batch* to limit concurrent Plotly rendering. Parity was enforced by having the API routers reuse the screener and peer modules rather than re-implementing SQL. Test assertions use *pytest.approx* with explicit tolerances, and golden files in *output/* serve as regression oracles.

d) Key Learnings and Insights

This module brought the system together and taught me that *delivery is a design concern*, not an afterthought. A dashboard that cannot be refreshed from a clean clone is a demo, not a product; an API that returns a different "latest year" than the UI is a bug, not a feature. I learned that automated reports are the most durable artifact: they survive even if the dashboard host is down, and they are the first thing a reviewer opens. Most importantly, I learned that QA is not about achieving "zero failures" but about making failures *visible and actionable* — the logs and HTML report are as much a deliverable as the PDFs. Completing the full loop from ETL to deployment gave me confidence that I can ship a complete analytics product, and the checklist at the end of Module 5 (environment variables, CORS, build artifacts, README) is a habit I will reuse for every future release.

TECHNOLOGIES

A stack of modern, open-source technologies was used to carry out the full analytics life cycle, from raw Excel ingestion to deployed interactive product. The choices reflect three principles: **reproducibility** (pinned versions, file-based outputs), **portability** (SQLite, pure Python, no external SaaS dependency for core analytics), and **clarity** (small, single-purpose modules rather than a monolithic notebook). The foundation of every analysis was **Python 3.11** with **Pandas** and **NumPy** for DataFrame and numeric work; **openpyxl** for Excel I/O; and **SQLite** for the warehouse.

Category	Technology	Role in Project	Key Files / Usage
Language	Python 3.11	Unifying language for ETL, analytics, dashboard, API	All *src/* modules
Data Processing	Pandas 3.0.5, NumPy 2.5.2, openpyxl 3.1.5, PyArrow 25.0.1	Excel reading, DataFrame transforms, type coercion, Excel export	processor.py, ratios.py, screener engine, reports
Database	SQLite (via sqlite3) + SQLAlchemy 2.0.52	Central warehouse, FK enforcement, queryable store	db/schema.sql, db_loader.py, nifty100.db
Visualization	Plotly 7.0.0, Matplotlib, Pillow 12.3.0	Interactive charts (dashboard) & static reports (sector, radar, heatmap)	reports/*.py, dashboard pages
Dashboard	Streamlit 1.63.0	8-module interactive web app, wide layout, cached queries	src/dashboard/app.py, pages/*.py
API	FastAPI, Uvicorn 0.52.4, Starlette, Python-multipart	REST API, CORS, OpenAPI docs, 8 routers	src/api/main.py, routers/*.py
Machine Learning	scikit-learn (KMeans, StandardScaler)	Unsupervised segmentation into 5 business archetypes	clustering.py, cluster_analysis.py
NLP	Python rule engine + Regex	Rule-based pros/cons extraction, confidence >60, 874 signals	nlp/parser.py, pros_cons_generator.py
Testing & QA	pytest 9.1.1, attrs, pluggy, pyarrow	Automated test suite, HTML report, regression checks	tests/, reports/pytest_report.html
Documentation	PDF (ReportLab), Markdown, OpenAPI JSON, Postman Collection	Analyst guide, data dictionary, API contracts	docs/analyst_guide_grand.pdf, data_dictionary.md
Version Control	Git & GitHub	Source control, branching, PRs, audit trail	Makefile, .gitignore, retrospectives
Environment	venv, pip, requirements.txt, Makefile	Reproducible setup, one-command ETL and loaders	requirements.txt, Makefile

Table 6: Technology stack — category, technology, role, and key usage in the project.

These technologies also rest on open standards, which makes the skills transferable. SQLite follows the SQL standard, Pandas and NumPy follow the Python data ecosystem, and the exchange of data relies on JSON and HTTP. Because the stack uses Python from the browser (via Streamlit) to the server (via FastAPI) to the warehouse, the same data structures, naming habits, and validation techniques were reused in every layer, reducing the cost of switching contexts and making it possible to see the application as one connected system.

Each technology also plays a role in reliability. Pandas' *errors="coerce"* protects the warehouse from malformed inputs, SQLite's *foreign_keys=ON* protects referential integrity, FastAPI's automatic validation protects the API from malformed requests, and pytest protects against regressions. Treating these measures as part of the design from the beginning, instead of adding them at the end, is a principle the internship reinforced in every module.

Several tools supported the workflow around the code. **Visual Studio Code** was the primary editor, with the Python and SQLite extensions ensuring a consistent style. **Git and GitHub** provided version control, branching, and collaboration. **Postman** was used to design, test, and document API requests, and **DB Browser for SQLite** was used to inspect stored tables. Finally, the reports were built to be host-agnostic — HTML, Excel, and PDF outputs require no server to view, ensuring that analytics remain accessible even offline. Quality was supported by habits that are not tied to any single tool: code was formatted, configuration was separated from code through YAML and environment variables, secrets were kept out of the repository, and every feature was tried with both valid and invalid input before it was committed.

Documentation was written as the work progressed, in the form of README files and comments for non-obvious decisions, because the best time to explain a decision is when it is still fresh.

Learning in this internship was also shaped by the tools that provide feedback. The **terminal** and **load_audit.csv** showed ingestion results immediately, **pytest** showed exactly which invariant failed, and the **Streamlit** and **FastAPI** logs showed what happened in the running product. Learning to read such feedback calmly, to form a hypothesis and to test it with the smallest possible change proved as important as learning any particular syntax, and it is a skill that applies to every programming language and framework.

The combination of these technologies illustrates an important principle: a complete analytical product is the result of many small, well-chosen tools that cooperate through clear interfaces. HTTP and JSON connect the client to the server, pandas connects the Excel to the warehouse, and Git connects the developer to the history of the project. Understanding each interface, instead of treating the stack as a black box, was one of the most valuable outcomes of the internship. The versions of the tools matter in practice: the project uses stable releases of Python, Pandas, Streamlit, and FastAPI, and the exact versions are recorded in *requirements.txt* so that the application can be installed again on any machine with the same result. Keeping dependencies updated, and reading the release notes before upgrading, was adopted as a regular maintenance practice.

The choice of the **Pandas + SQLite + Streamlit + FastAPI** stack also reflects practical considerations. All four main technologies are open source and have large communities, extensive documentation, and a rich ecosystem of packages, so that solutions to common problems are easy to find. JSON is used as the format of data from the warehouse to the browser, which removes the need to convert between different representations. For a beginner, the stack offers a short path from an idea to a working and deployed product, while remaining powerful enough to be used for real analytics at the scale of the Nifty 100. For a production handover, the same stack can be extended with Docker, CI/CD via GitHub Actions, and cloud deployment without rewriting the analytics — a path already sketched in the project's "Future Enhancements" section and in *docs/sprint6_retrospective.md*.

CONCLUSION

The Data Analyst Internship at **Bluestock Fintech** under the guidance of **Mr. Yash Kale (Project Manager)** was a complete and rewarding learning experience. Over the period from **24 July 2026 to 24 September 2026** (Certificate ID **BFDA108893**, dated 24/09/2026), the internship guided the progression from the raw structure of financial disclosures to the deployment of an interactive, API-served analytics product. Each module built systematically upon the previous one, ensuring a smooth and logical movement from the foundations of ETL and warehouse design, through the computation of financial ratios and the construction of screening and peer engines, to valuation, cash-flow intelligence, NLP-based signals, machine-learning segmentation, and finally to the delivery of a Streamlit dashboard and FastAPI service with automated reporting and testing. The internship demonstrated that financial analytics is not simply the knowledge of several ratios or the ability to call a clustering algorithm, but the ability to understand how data, models, and delivery layers work together. The ETL pipeline, the 12-table SQLite warehouse, the ratio engine, the six screener presets, the peer-percentile framework, the valuation flags, the cash-flow KPIs, the 874 NLP signals, the five KMeans archetypes, the 8-module dashboard, the 8-router API, and the 91 tearsheets together form a coherent, working product. The challenges encountered along the way — inconsistent headers, year normalization, financial-sector leverage handling, sparse NLP sources, report scale, and dashboard–API parity — strengthened analytical thinking and developed a systematic approach to debugging that values audit trails over ad-hoc fixes. Beyond technical skills, the hybrid mode of the internship encouraged self-discipline, time management, and independent learning, qualities that are essential for every professional analyst. Habits such as writing auditable code, committing work in small, well-described steps, documenting decisions alongside code, and testing before delivery were reinforced throughout the programme. The certificate and the verifiable repository together formally recognize this effort, and the artifacts remain executable by any reviewer who clones the repository and follows the README. Looking ahead, the experience gained during this internship opens several directions for further growth. Adding real-time market-data integration would remove the current reliance on static Excel extracts; automated scheduled data refresh would make the warehouse live; additional valuation models (DCF, residual income) would deepen the fundamental toolkit; historical time-series dashboards with anomaly detection would move from cross-sectional to longitudinal analytics; advanced portfolio optimization would translate screens into backtests; more sophisticated NLP models (transformer-based sentiment or thematic extraction) would enrich the signal set; explainable clustering (e.g., SHAP for cluster membership) would make archetypes more actionable; user authentication and cloud deployment (Docker, CI/CD with GitHub Actions) would make the product multi-tenant and continuously delivered. The project's "Future Enhancements" section in the analyst guide already sketches this roadmap. In conclusion, the internship achieved its objective of bridging the gap between academic knowledge and industry practice in financial data analytics. It has given the author a strong practical foundation, a portfolio of working analytics and a deployed product, and the confidence to continue learning advanced topics such as time-series forecasting, causal inference, explainable AI, cloud architecture, and financial modeling. These skills will contribute significantly to the author's future career as a data analyst and financial-analytics engineer, and the discipline of treating automated outputs as decision-support requiring human review will remain a guiding principle.

Key project metrics at a glance:

Metric	Value	Source
Companies in master	92	data/processed/companies.csv
Financial ratio rows	1,160	financial_ratios
NLP signals (pros/cons)	874 (562/312)	pros_cons_generated.csv
Clusters (KMeans, k=5)	5 archetypes	cluster_labels.csv
Tearsheets / Radar charts	91 / 89	reports/tearsheets/, radar_charts/

Metric	Value	Source
Sector reports	10	reports/sector/
API routers	8	src/api/routers/
Dashboard modules	8	src/dashboard/pages/

REFERENCES

[1] Bluestock Fintech, “Certificate of Internship — Bellamkonda Mokshith (BFDA108893),” verification QR, 24 Sep 2026. [Online]. Available: verification portal via QR on certificate. [Accessed: Sep. 2026].

[2] Mohan Babu University, “Internship Report Guidelines — Department of Data Science, School of Computing, Tirupati,” 2025–2026.

[3] Nifty 100 Analytics Dashboard Repository, mokshith005/NIFTY100 (branch arena/b832ef02-nifty100), “README.md, docs/analyst_guide_grand.pdf, docs/data_dictionary.md,” 2026. [Online]. Available: GitHub repository.

[4] Wes McKinney, “pandas: powerful Python data analysis toolkit,” pandas documentation. [Online]. Available: https://pandas.pydata.org/docs/. [Accessed: Sep. 2026].

[5] SQLite Consortium, “SQLite Documentation — Foreign Keys, Datatypes, SQL Syntax,” [Online]. Available: https://www.sqlite.org/docs.html. [Accessed: Sep. 2026].

[6] Streamlit Inc., “Streamlit Documentation — Build data apps in Python,” [Online]. Available: https://docs.streamlit.io/. [Accessed: Sep. 2026].

[7] Sebastián Ramirez, “FastAPI Documentation — Modern, fast web framework,” [Online]. Available: https://fastapi.tiangolo.com/. [Accessed: Sep. 2026].

[8] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://scikit-learn.org/.

[9] D. J. Hand, H. Mannila, and P. Smyth, Principles of Data Mining, Cambridge, MA, USA: MIT Press, 2001.

[10] A. Damodaran, Investment Valuation: Tools and Techniques for Determining the Value of Any Asset, 3rd ed., Hoboken, NJ, USA: Wiley, 2012.

[11] B. G. Malkiel, A Random Walk Down Wall Street, New York, NY, USA: W. W. Norton, 2019.

[12] Python Software Foundation, “Python 3.11 Documentation,” [Online]. Available: https://docs.python.org/3.11/. [Accessed: Sep. 2026].

[13] Plotly Technologies Inc., “Plotly Python Graphing Library,” [Online]. Available: https://plotly.com/python/. [Accessed: Sep. 2026].

[14] K. Chodorow, MongoDB: The Definitive Guide (for comparison with SQLite document model), Sebastopol, CA, USA: O’Reilly Media, 2019.

[15] S. Chacon and B. Straub, Pro Git, 2nd ed., New York, NY, USA: Apress, 2014.

[16] World Wide Web Consortium, “CSS Flexible Box Layout — report styling reference,” [Online]. Available: https://www.w3.org/TR/css-flexbox-1/. [Accessed: Sep. 2026].

[17] J. A. Hartigan and M. A. Wong, “A K-means clustering algorithm,” Applied Statistics, vol. 28, no. 1, pp. 100–108, 1979.

[18] E. Hull, K. Jackson, and J. Dick, Requirements Engineering, 3rd ed., London, UK: Springer, 2011 — referenced for ETL requirements traceability.

APPENDICES

Appendix A — Repository File Map (selected). *data/raw/core/*.xlsx (7), data/raw/supplementary/*.xlsx (5), data/processed/*.csv (12), db/schema.sql, src/etl/*.py (6), src/analytics/*.py (8), src/screener/engine.py, src/nlp/*.py (2), src/dashboard/app.py + pages/*.py (8), src/api/main.py + routers/*.py (8), src/reports/*.py (7), output/*.csv|xlsx (15+), reports/tearsheets/*.pdf (91), reports/radar_charts/*.png (89), reports/sector/*.pdf (10), docs/analyst_guide_grand.pdf, data_dictionary.md, openapi.json, pytest_report.html.*

Appendix B — Reproduction Commands.

```
processor: python -m src.etl.processor
loader: python -m src.etl.db_loader
ratios: python -m src.analytics.populate_ratios
screener: python -m src.screener.engine
peer: python -m src.analytics.peer
valuation: python -m src.analytics.valuation
cashflow: python -m src.analytics.cashflow_kpis
nlp: python -m src.nlp.pros_cons_generator
clustering: python -m src.analytics.clustering
dashboard: streamlit run src/dashboard/app.py
api: uvicorn src.api.main:app --reload
tests: pytest -v --html=reports/pytest_report.html --self-contained-html
```

Appendix C — Data Quality Exceptions. ATGL and SBIN have no financial-ratio rows in the current build (see *output/ratio_edge_cases.log* and *output/load_audit.csv*). These are retained as explicit exceptions, not imputed. All other companies have at least one ratio row; coverage is 1,160 rows across 92 companies.

Appendix D — Certification. Internship certificate is reproduced on page iii (ID BFDA108893) and is verifiable via QR code. This report aligns with Mohan Babu University formatting (margins 1 inch, Times family, justified body, centered headings, dotted TOC) and documents the Bluestock Fintech Data Analyst Internship under Mr. Yash Kale.